

Introduction to Neural Networks

Aijun An

Department of Electrical Engineering and Computer Science

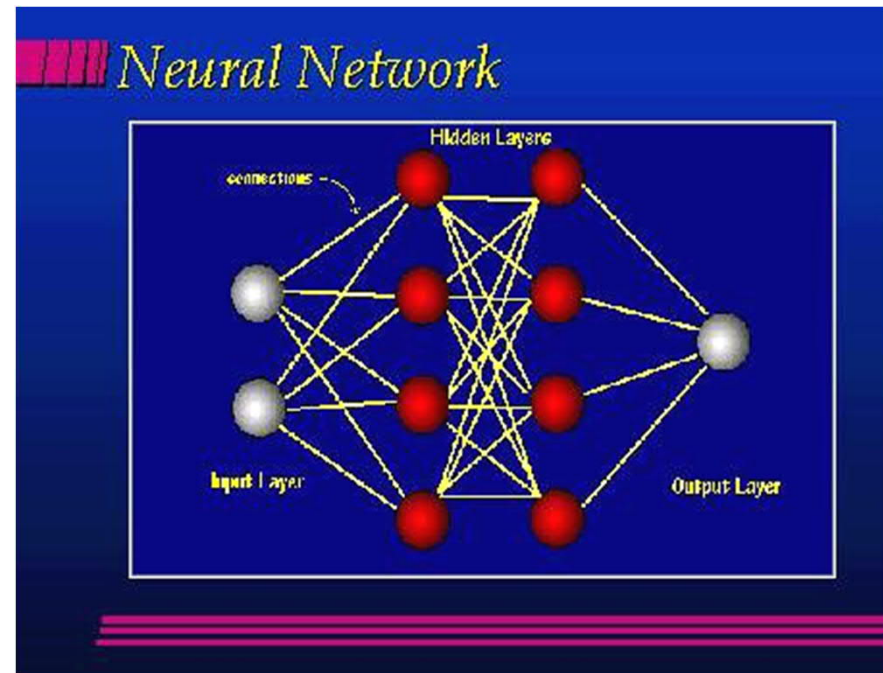
York University

Outline

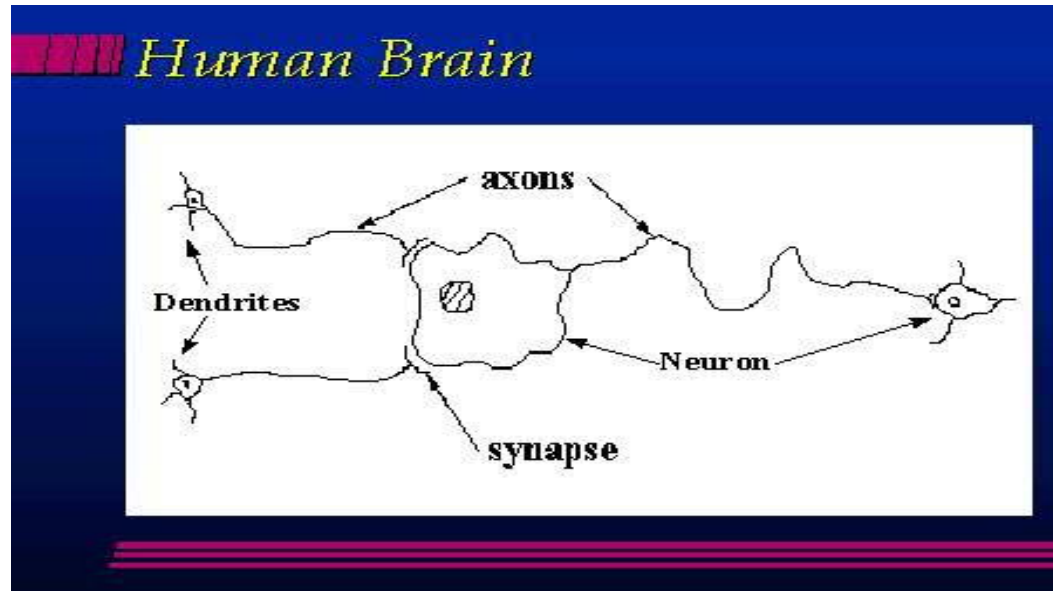
- ▶ Introduction
- ▶ Types of general neural network structures
- ▶ What happens inside a neuron?
- ▶ Activation Functions
- ▶ Output and Input Design
- ▶ How to train a neural network
 - ▶ Backpropagation algorithm
- ▶ How to prevent overfitting
- ▶ Applications and history of neural networks
- ▶ Brief introduction to deep learning

What is an Artificial Neural Network?

- ▶ It is a formalism for representing functions inspired from biological systems
- ▶ It consists of layers of interconnected computing units
 - ▶ Each unit computes a simple function.
 - ▶ The inputs of the units in one layer are typically the outputs of the units in the previous layer.
 - ▶ Each connection is associated with a weight.
 - ▶ Parallel computing can be performed among the units in each layer

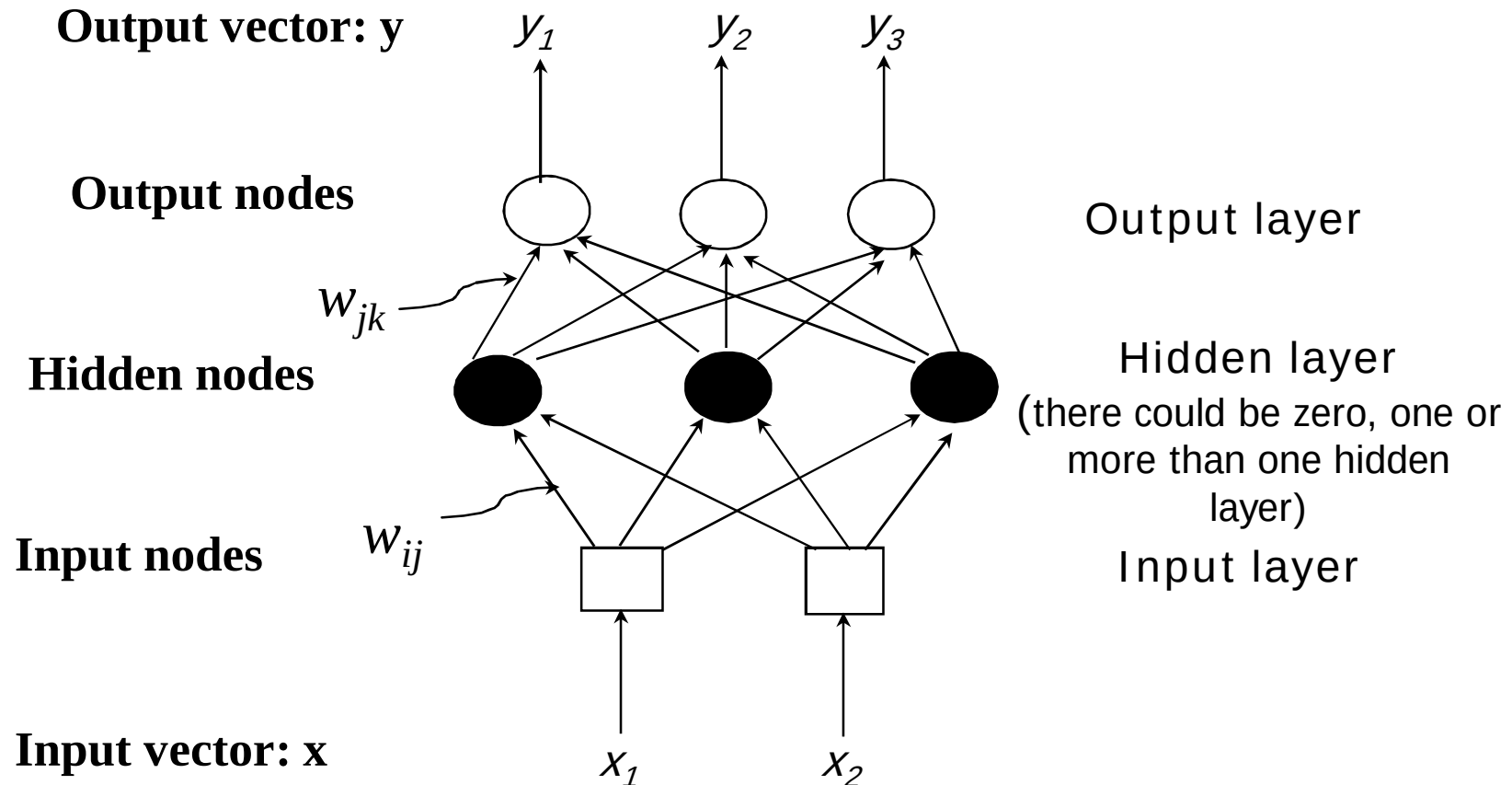


Biological Motivation



- ▶ Biological Learning Systems are built of very complex webs of interconnected neurons.
 - ▶ Number of neurons: 10^{10}
 - ▶ Connections per neuron: 10^4 -- 10^5
- ▶ Information-Processing abilities of biological neural systems must follow from highly *parallel processes* operating on representations that are distributed over many neurons
 - ▶ Scene recognition time: 0.1 second
- ▶ ANNs attempt to capture this mode of computation

An Example of Neural Networks



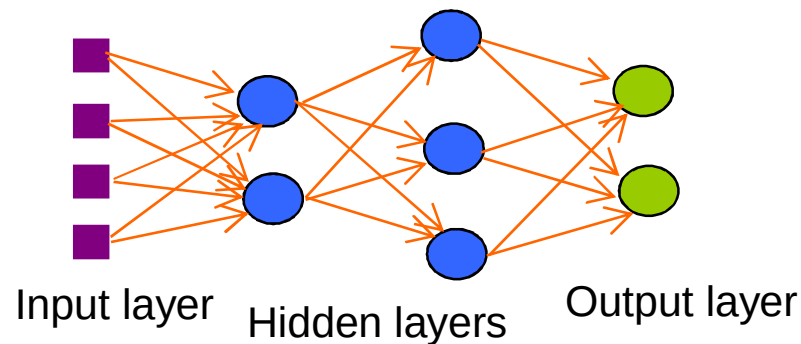
- ▶ Each hidden or output node or unit is called a neuron
- ▶ Each connection is associated with a weight
- ▶ The network is usually **fully connected**, unless otherwise specified.
 - ▶ Each unit provides an input to each unit in the next layer

Types of Neural Networks

- ▶ *Feed-forward* neural networks

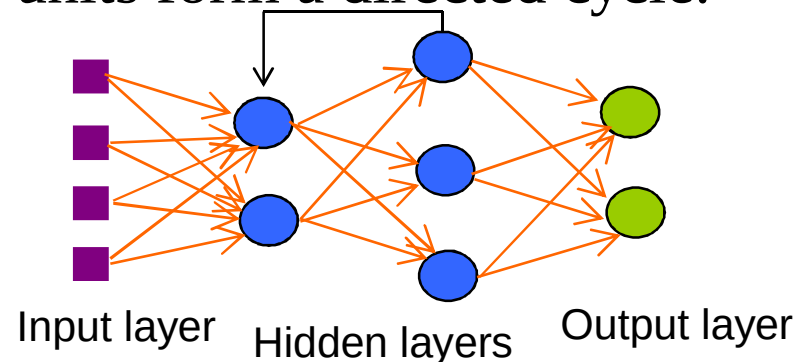
- ▶ None of the connections cycles back to an input unit or a unit of a previous layer.

- ▶ An example:



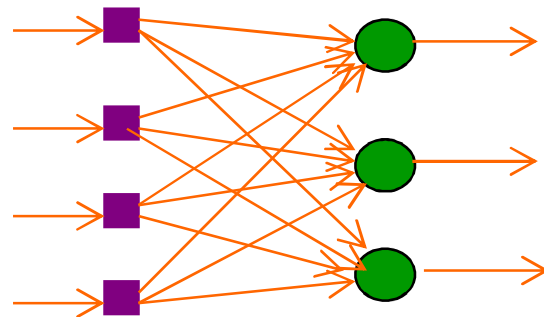
- ▶ *Recurrent* neural networks (*Feedback* neural network)

- ▶ Connections between units form a directed cycle.



Single-layer Feed-Forward Neural Network

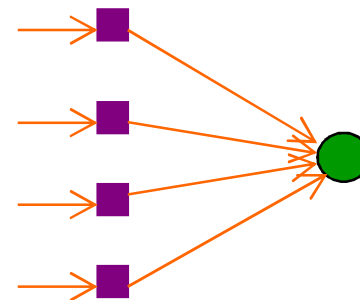
- ▶ A single-layer network does not have hidden layers



Input layer Output layer

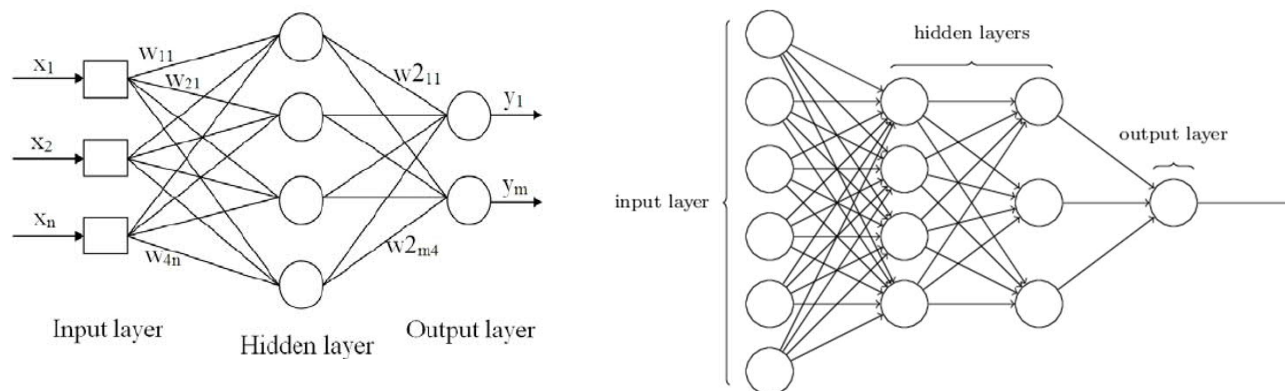
- ▶ The input layer is not counted as a “layer” because no computation is performed there.
- ▶ Perceptron
 - ▶ Contain only one computing node

If the *sigmoid* activation function is used in the perceptron, it is the **logistic regression** model



Multi-layer Feed-Forward Neural Network

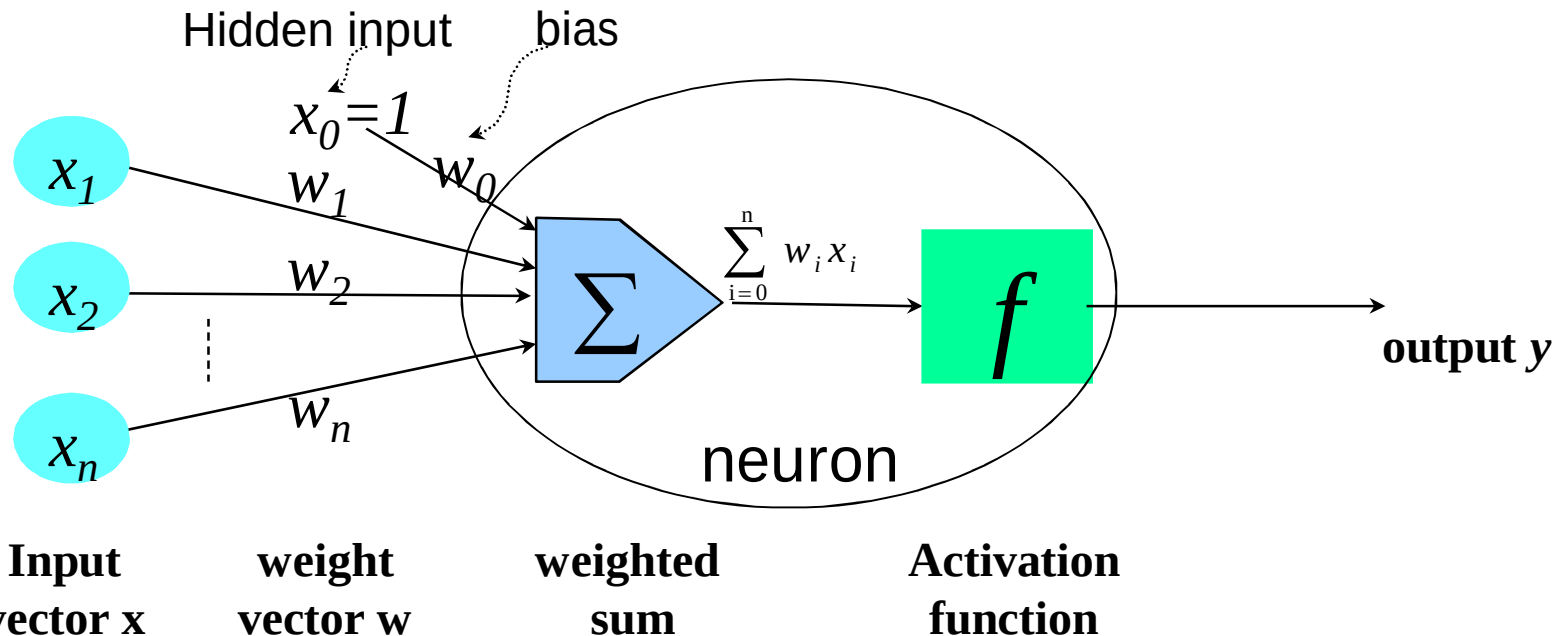
- ▶ A multilayer network contains one or more hidden layers.
 - ▶ It overcomes the limitation of single-layer NN: single-layer NN cannot handle non-linearly separable learning tasks.



- ▶ If the network is *fully connected*, it is called **Multilayer Perceptron (MLP)**
- ▶ When the number of hidden layers > 1 , it is considered as a **deep neural network (DNN)**.
 - ▶ A DNN may or may not be fully connected

What Happens inside a Neuron?

- ▶ A neuron is the basic information processing unit of an NN



- ▶ A neuron first sums the weighted inputs and then apply an activation function
- ▶ *Activation function*
 - ▶ Often maps a large input domain onto a smaller range (if so, it is called *squashing function*)
 - ▶ Often nonlinear (if so, referred to as *nonlinearity*)

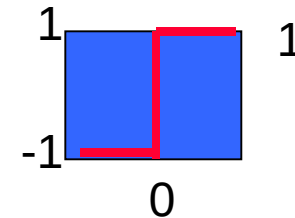
Examples of Activation Functions

- ▶ Threshold logic function
- ▶ Sigmoid function
- ▶ Tanh function
- ▶ Relu function
- ▶ Leaky Relu
- ▶ ...

Examples of Activation Functions

- ▶ Threshold logic function, such as,

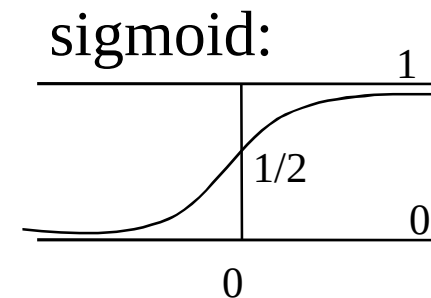
$$f(x) = \text{sign}(x) = \begin{cases} 1 & x > 0 \\ -1 & \text{otherwise} \end{cases}$$



- ▶ not a smooth function
- ▶ not usually used in modern neural networks

- ▶ Sigmoid function:

$$f(x) = \frac{1}{1 + e^{-x}}$$



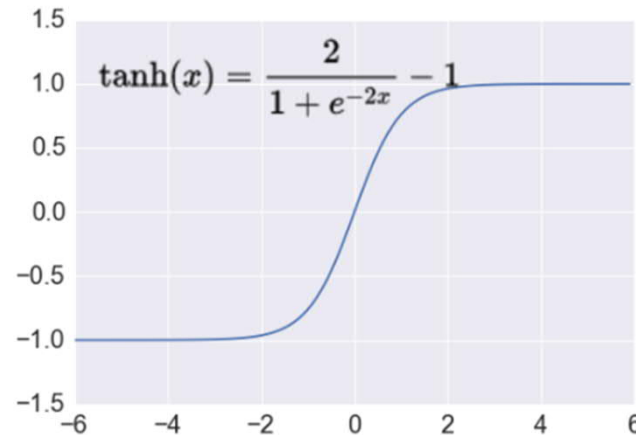
- ▶ differentiable:

$$f'(x) = f(x)(1 - f(x))$$

a good feature for the purpose of training neural networks

Examples of Activation Functions

- ▶ Tanh function:

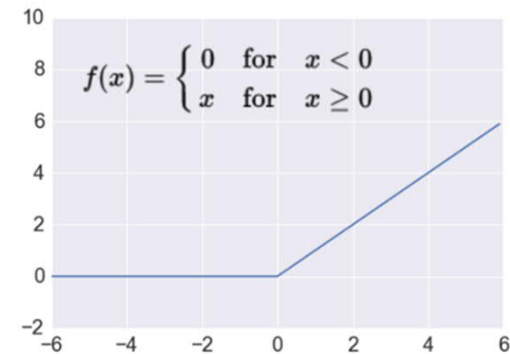


- ▶ Tanh is a scaled sigmoid: $\tanh(x) = 2\text{sigmoid}(2x) - 1$
- ▶ Range: $(-1, 1)$
- ▶ Differentiable.
- ▶ Advantage over sigmoid: larger derivative
 - ▶ For input range $[-1, 1]$, the derivative of Tanh ranges $[0.42, 1]$, while the derivative of Sigmoid ranges $[0.20, 0.25]$.
 - ▶ Zero-centered, making the optimization process more flexible and faster to converge

Examples of Activation Functions

- ▶ Rectified Linear Units (ReLU),

$$f(x) = \max(0, x)$$



- ▶ Commonly used in deep neural networks

- ▶ Differentiable:

$$f'(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

Cavet: Dying Relu problem:

- Could produce **dead neurons** that outputs zero for all training examples

- ▶ Advantages:

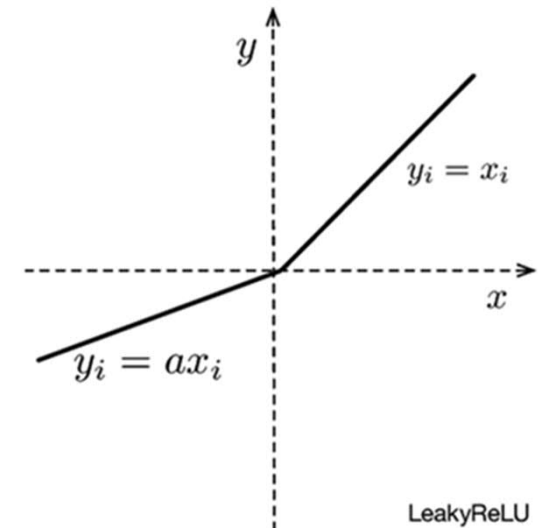
- ▶ Efficient computation
- ▶ No vanishing gradient problem in network training
- ▶ Sparse activation (many neurons output zero), which helps prevent overfitting.

Examples of Activation Functions

- ▶ Leaky ReLU:

$$f(x) = \max(\alpha x, x)$$

where α is a small value, e.g., 0.01



- ▶ Commonly used in deep neural networks

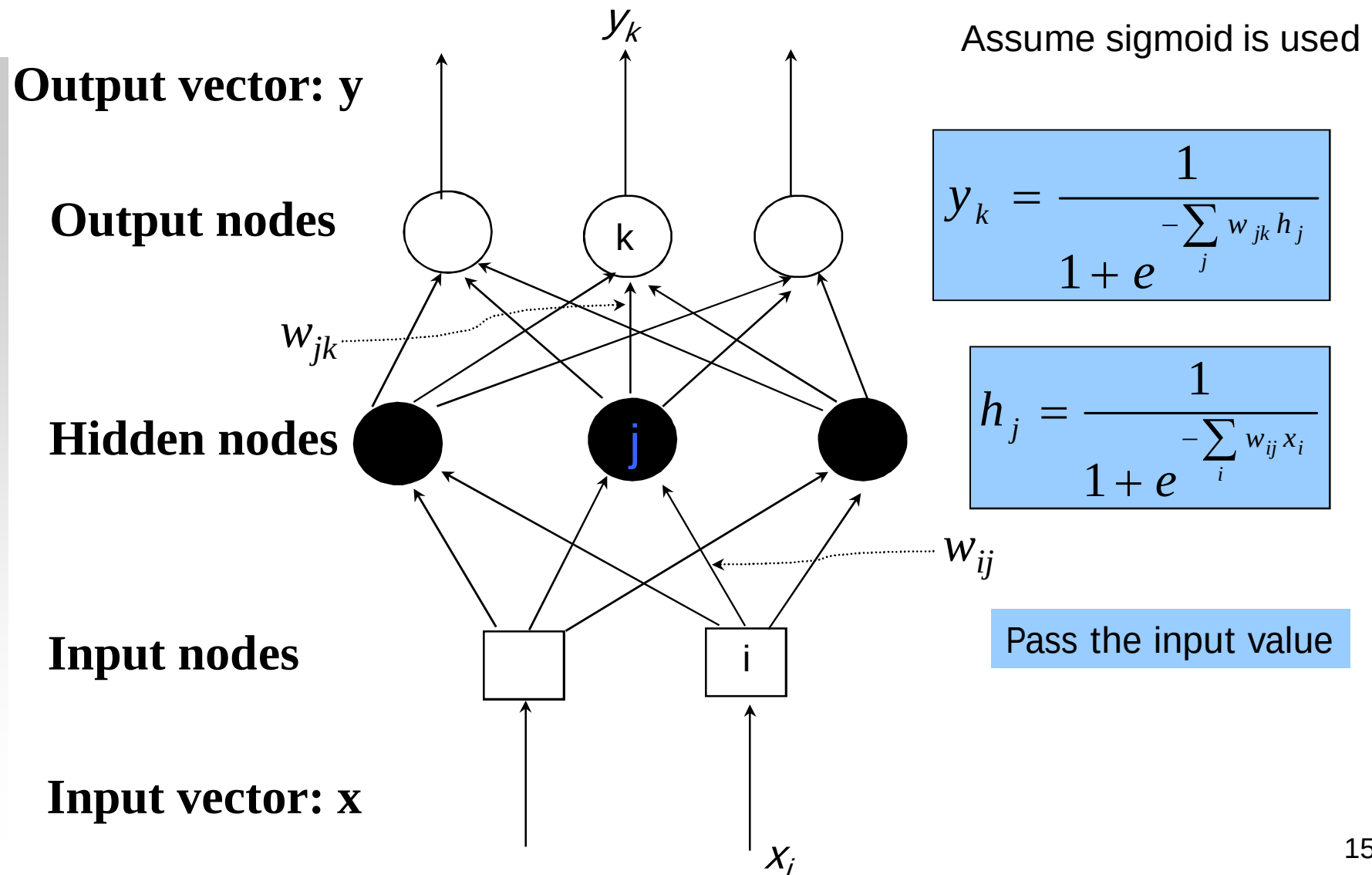
- ▶ Differentiable:

$$f'(x) = \begin{cases} 1, & \text{if } x > 0 \\ \alpha, & \text{otherwise} \end{cases}$$

- ▶ Advantages:

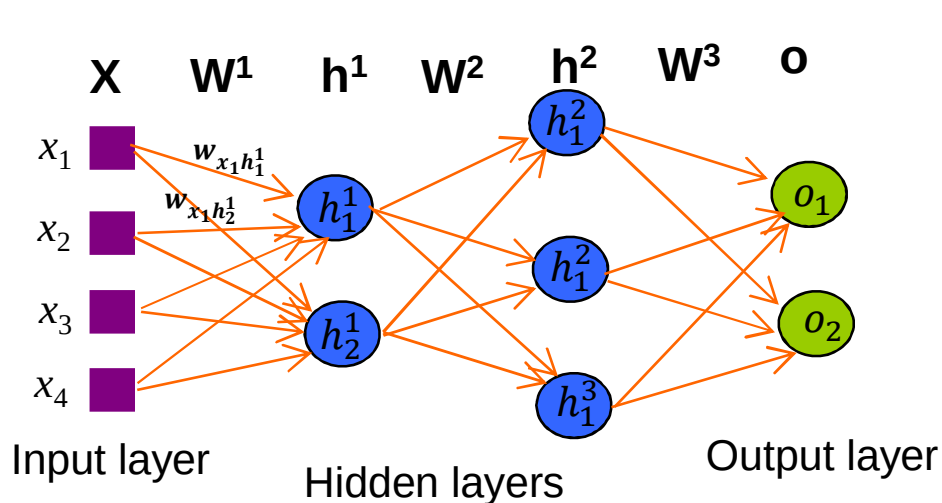
- ▶ Similar to ReLU but with less sparsity
- ▶ Solving the dying Relu problem

How Output is Computed?



How Output is Computed?

- Computation can be expressed using matrix multiplications:



W ¹	x ₁	x ₂	x ₃	x ₄
h ₁ ¹	0.4	0.3	-0.1	0.5
h ₂ ¹	0.1	-0.8	0.2	-0.1

Diagram illustrating the weight matrix W¹ and the input vector x. The matrix W¹ is shown with rows h₁¹ and h₂¹, and columns x₁, x₂, x₃, x₄. The input vector x is shown with elements x₁, x₂, x₃, x₄. The output of the first hidden layer is shown as h₁¹ and h₂¹. The weights w_{x₁h₁¹} and w_{x₁h₂¹} are indicated by arrows pointing to the corresponding elements in the matrix.

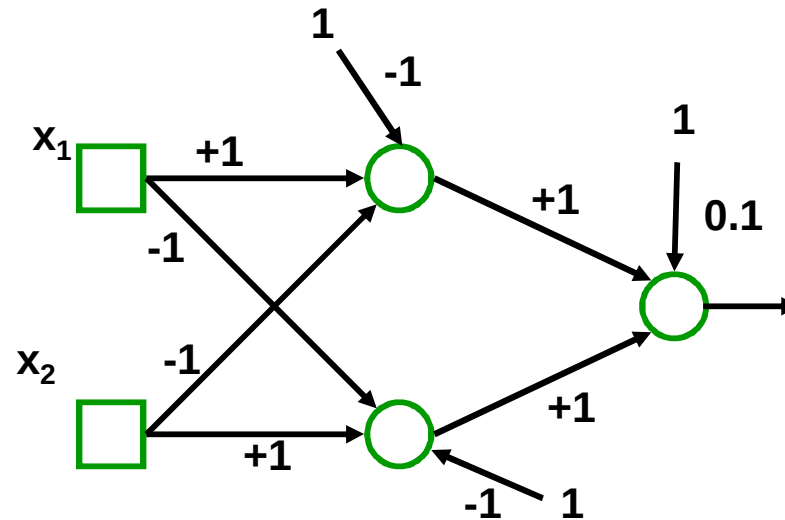
$$\mathbf{h}^1 = f(\mathbf{W}^1 \mathbf{x}) \quad \mathbf{h}^2 = f(\mathbf{W}^2 \mathbf{h}^1) \quad \mathbf{o} = f(\mathbf{W}^3 \mathbf{h}^2)$$

where f is the activation function.

Function Modeled by a Neural Network

- ▶ A neural network can be used to model a function:
 - ▶ $y=f(\mathbf{x})$ where $\mathbf{x}$ is the input vector and $\mathbf{y}$ is the output vector
- ▶ The function it models is determined by
 - ▶ Topology of the neural network:
 - ▶ the number of input units in the input layer
 - ▶ the number of hidden layers
 - ▶ the number of units in each hidden layer
 - ▶ the number of units in the output layer
 - ▶ Values of weights
 - ▶ Activation function chosen.

What function does this NN model?



- ▶ x_1 and x_2 takes binary values of $\{-1, 1\}$
- ▶ Activation function is:

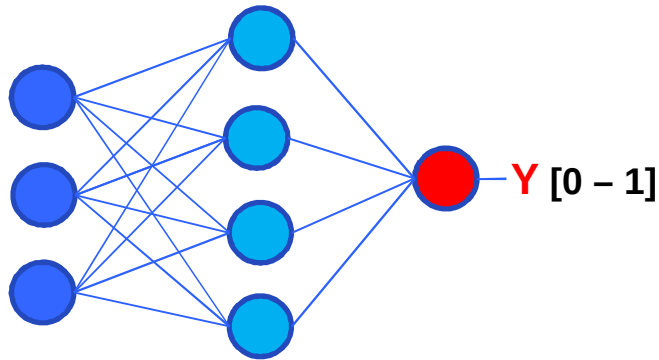
$$f(x) = \text{sign}(x) = \begin{cases} 1 & x > 0 \\ -1 & \text{otherwise} \end{cases}$$

Choice of Activation Function

- ▶ Hidden layer nodes:
 - ▶ Usually use the same activation function for all hidden nodes.
 - ▶ Common choices:
 - ▶ Multilayer perceptron (MLP): ReLU
 - ▶ Convolutional neural network (CNN): ReLU
 - ▶ Recurrent neural network (RNN): Sigmoid or Tanh
- ▶ Output layer nodes:
 - ▶ Depending on the problem type
 - ▶ Binary classification: Sigmoid
 - ▶ Multiclass classification: Softmax
 - ▶ Regression: linear (“identify” function or no activation)

Output Layer Design

- For binary classification:



Number of output nodes	1
Activation Function	Sigmoid
Class 1	$Y > 0.5$
Class 2	$Y < 0.5$

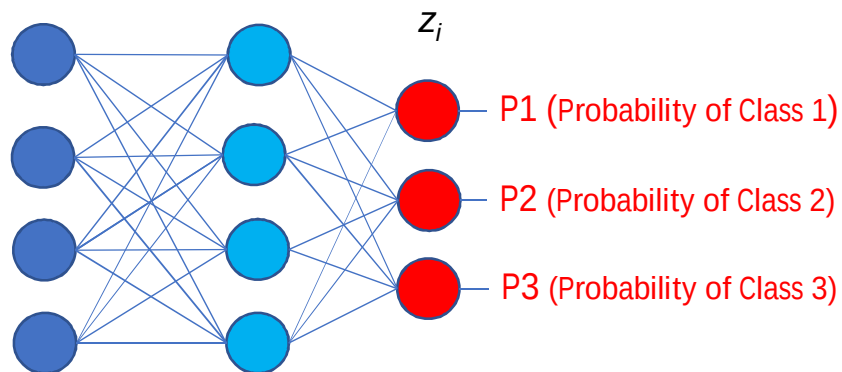
Activation function: $f(z) = \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$

Output Layer Design

► Multi-class classification

Age	Spectacle-prescrip	astigmatism	Tear-prod-rate	Contact-lenses
18	myope	no	reduced	none
20	hypermetrope	no	normal	soft
39	myope	yes	normal	hard

Number of output nodes	= number of classes
Activation Function	Softmax
Class	= argmax(P1, P2, P3)



z_i is the weighted sum of inputs to the i th output node (before activation)

Desired output for each output node:

none	soft	hard
1	0	0
0	1	0
0	0	1

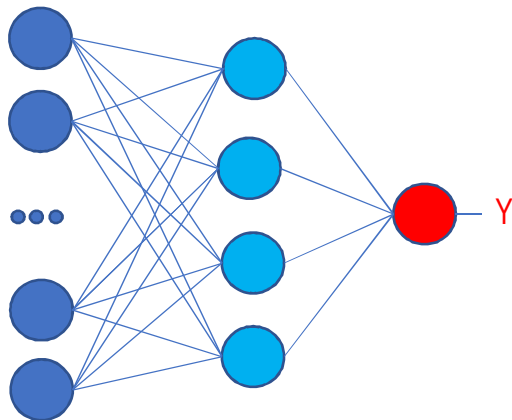
$$\text{Softmax: } P_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \sum_{j=1}^K P_j = 1$$

Output Layer Design

► Regression

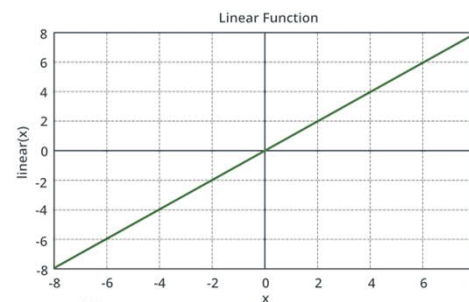


Features (independent variables)						Output (dependent variable)
abandon	ability	able	...	youth	zone	Number of Likes
5	0	10		0	0	1000
0	17	0		0	2	45
4	0	0		22	1	530



$$f(z) = z$$

Number of output nodes	= one
Activation Function	Linear (None)



Inputs to Neural Networks

- ▶ Neural net can take continuous input variables naturally.
- ▶ A discrete attribute can be encoded by:
 - ▶ using one input unit per value of the attribute
 - ▶ For example, if the domain of attribute A is {a1, a2, a3}
 - ▶ We assign 3 input units to represent A.
 - ▶ If A=a1, the input to the 3 units is 1, 0, 0.
 - ▶ If A =a2, the input to the 3 units is 0, 1, 0.
 - ▶ If A=a3, the input to the 3 units is 0, 0, 1.
- ▶ The input values to a neural net are usually normalized to speed up the learning phase.

*One-hot
encoding*

Training a Neural Network

- ▶ Before training can begin, the user must decide on
 - ▶ Network topology
 - ▶ # of hidden layers and # of units in each hidden layer
 - ▶ Type of the activity function
 - ▶ no clear rules on the best topology
 - ▶ network design is often a trial-and-error process
- ▶ The objective of training
 - ▶ search in the space of sets of weights to obtain a set of weights that makes almost all the examples in the training data classified correctly.
- ▶ Also called *connectionist* learning
- ▶ Most popular neural network learning algorithm is *backpropagation*

Training: Backpropagation algorithm

- ▶ Objective:

- ▶ searches for weight values that minimize a loss function over a set of training examples

- ▶ Loss function:

- ▶ Classification problem: **cross-entropy loss**

$$L(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K (y_k^i \log \hat{y}_k^i)$$

- ▶ θ denotes the weights
- ▶ K is the number of classes and N is the number of examples
- ▶ y^i is the true target value (1 or 0) of the i th example
- ▶ $\hat{y}^i$ is the predicted value for the i th example
- ▶ Regression problem: **mean squared error loss**

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (y^i - \hat{y}^i)^2$$

Cross-Entropy

- Cross-entropy measures the difference between two probability distributions p and q over a random variable V :

$$H(p, q) = - \sum_{v \in V} p(v) \log q(v)$$

- The higher the cross entropy, the more dissimilar between p and q .
- For example,
 - If $p=[0, 1, 0]$ and $q=[0.15, 0.6, 0.25]$, then

$$H(p, q) = -[0 \cdot \log(0.15) + 1 \cdot \log(0.6) + 0 \cdot \log(0.25)] = 0.736$$

- If $p=[0, 1, 0]$ and $q=[0.1, 0.8, 0.1]$, then

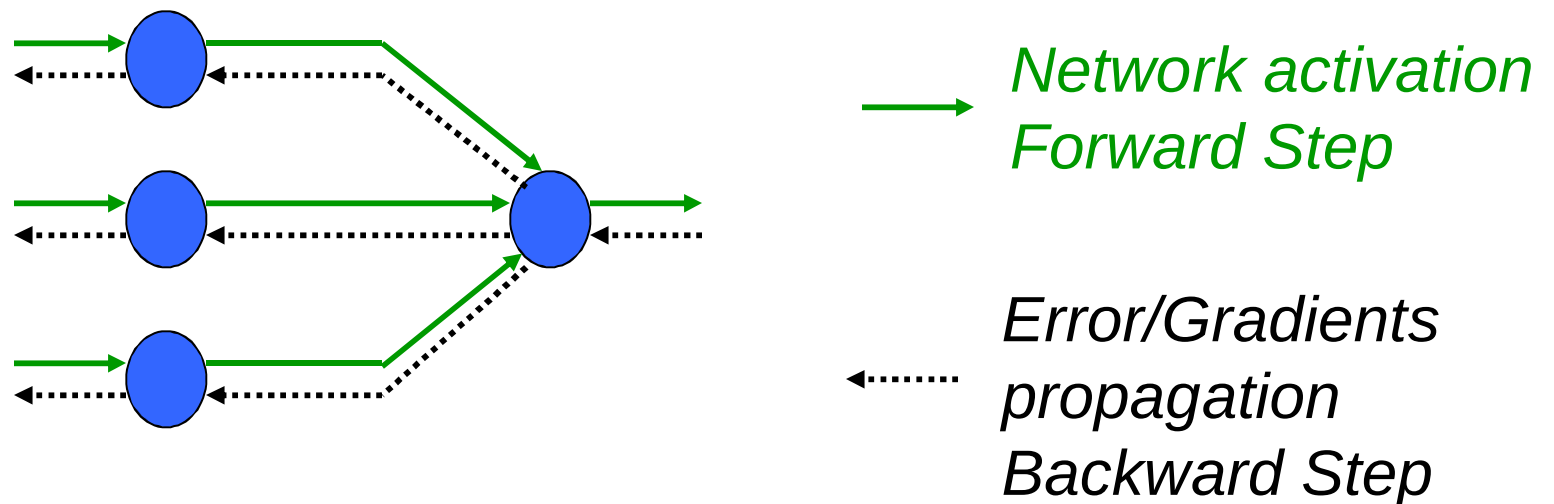
$$H(p, q) = -[0 \cdot \log(0.1) + 1 \cdot \log(0.8) + 0 \cdot \log(0.1)] = 0.2231$$

Training: Backpropagation algorithm

- ▶ The algorithm first initializes the weights with small random values
- ▶ Then backpropagation consists of the repeated application of the following two passes:
 - ▶ **Forward pass**: in this step the network is activated on one example and the output of the network is computed.
 - ▶ **Backward pass**: in this step, starting at the output layer,
 - ▶ the error of the output layer is computed
 - ▶ its gradients are propagated backwards through the network, layer by layer, and weights are updated.

Backpropagation

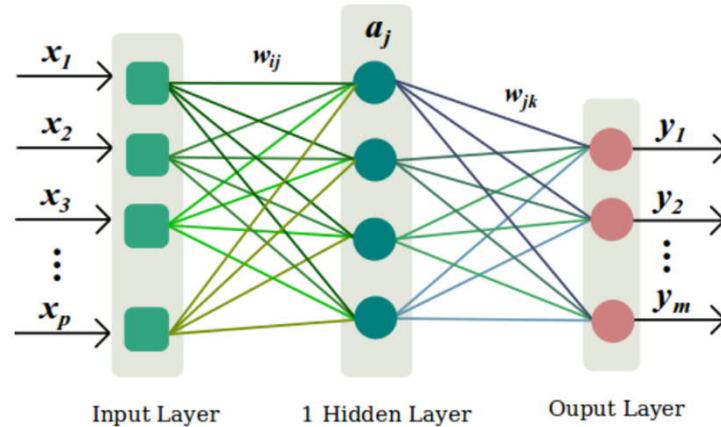
- ▶ Back-propagation training algorithm



- ▶ Backprop adjusts the weights of the NN in order to minimize the loss function.
- ▶ *Gradient descent* or *stochastic gradient descent* algorithm can be used

How to Update the Weights with Gradient Descent?

- $L(\theta)$ is a function of the weights $\theta = \{w_1, w_2, \dots\}$

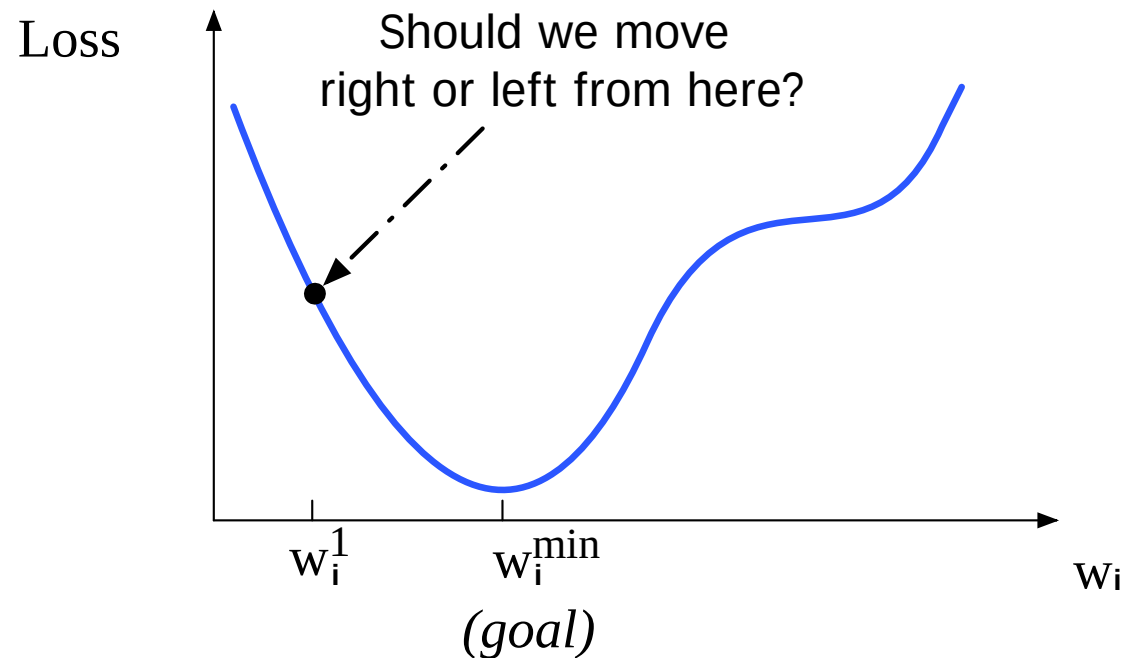


- Each weight w_i is updated by the value of the partial gradient $\frac{d}{dw_i} L(\theta)$ weighted by a **learning rate** η :

$$w_i = w_i - \eta \frac{d}{dw_i} L(\theta)$$

Higher learning rate η means move w_i faster, but if too big may keep oscillating

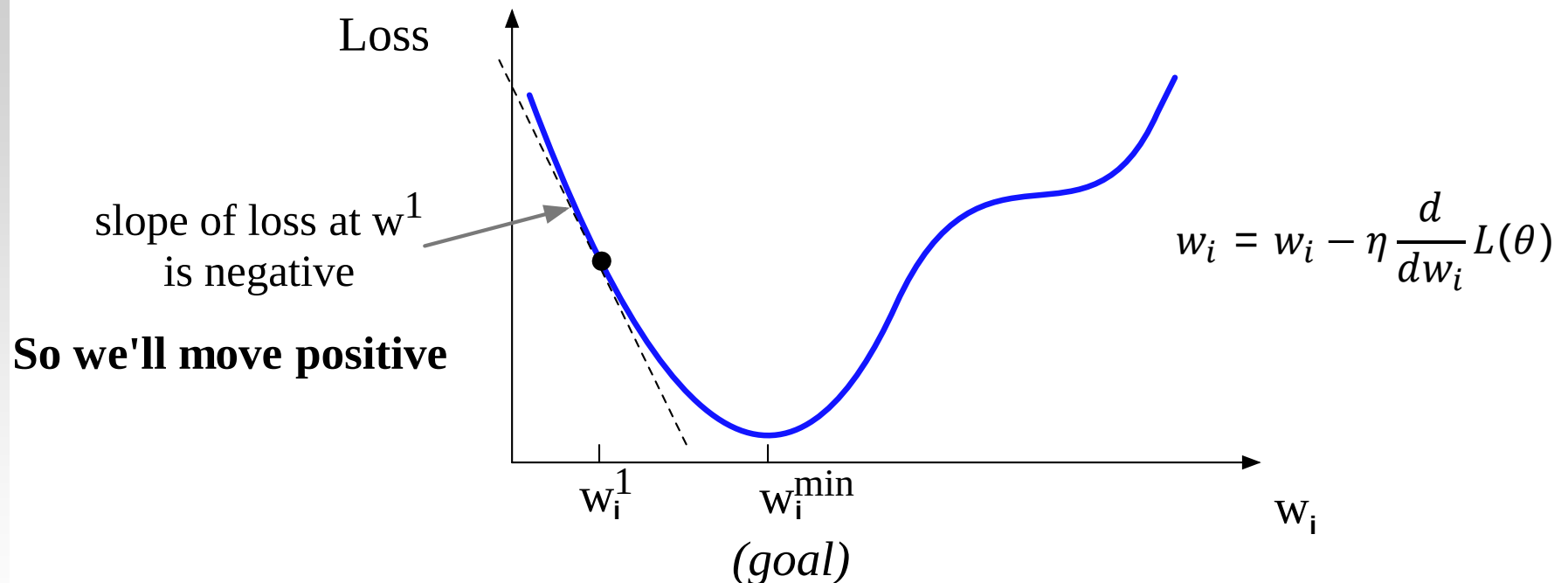
Why update weight this way?



(Picture credits: Daniel Jurafsky & James H. Martin, Speech and Language Processing)

Why update weight this way?

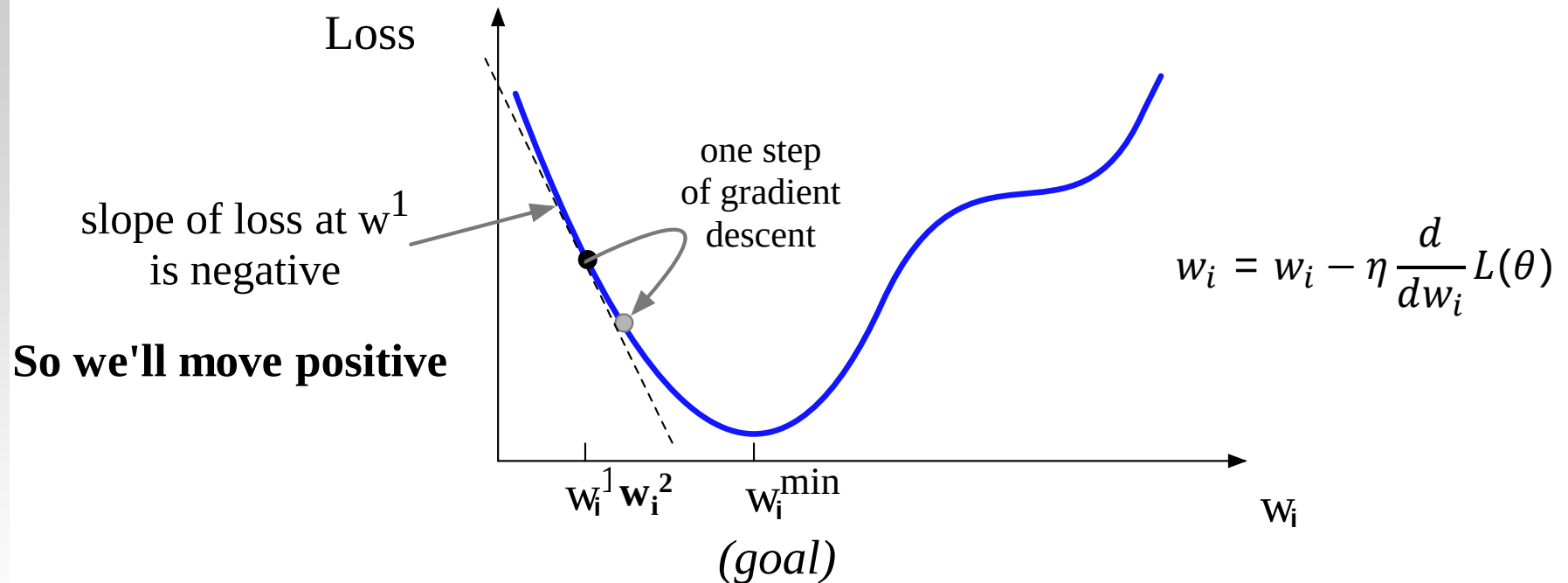
Find the gradient of the loss function at the current point and move in the opposite direction.



(Picture credits: Daniel Jurafsky & James H. Martin, Speech and Language Processing)

Why update weight this way?

Move w in the reverse direction from the slope of the function



(Picture credits: Daniel Jurafsky & James H. Martin, Speech and Language Processing)

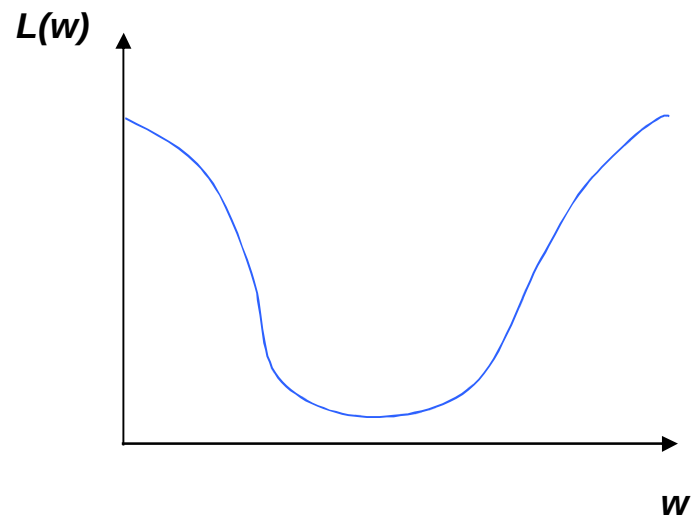
How much do we move in that direction ?

- The value of the gradient $\frac{d}{dw} L(\theta)$ weighted by a **learning rate η**

$$w_i = w_i - \eta \frac{d}{dw_i} L(\theta)$$

- ▶ Smaller η means moving w_i slower
- ▶ Higher η means moving w faster, but if too big may keep oscillating

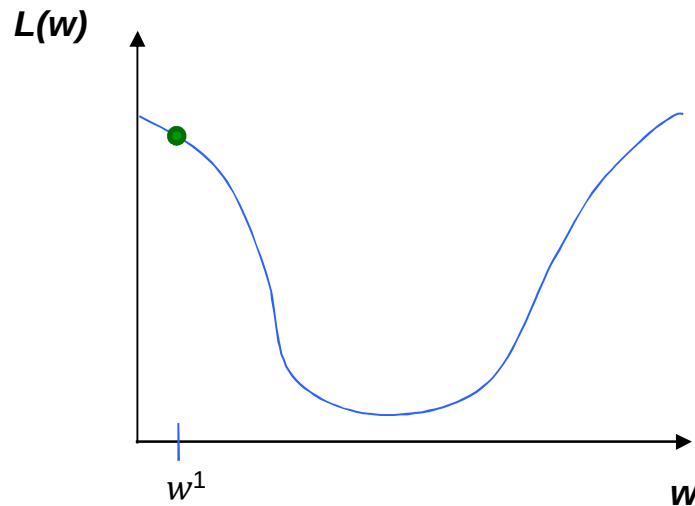
Illustration of Gradient Descent



(Adapted from James Tompkin (Brown University))

Illustration of Gradient Descent

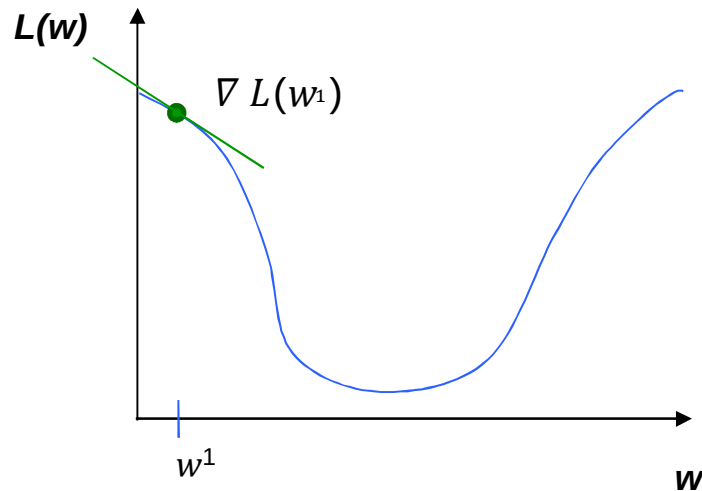
Pick random starting point.



(Picture credit: James Tompkin (Brown University))

Illustration of Gradient Descent

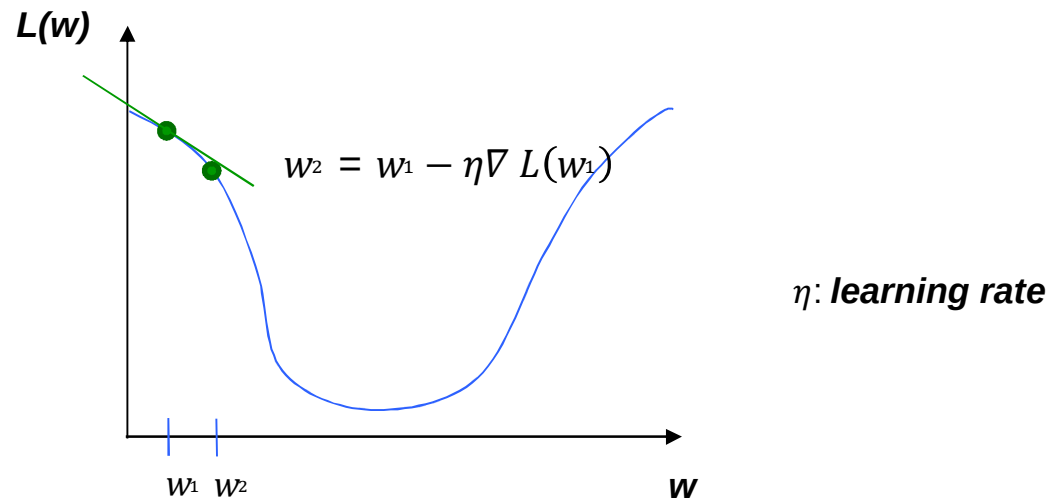
Compute gradient at point (analytically or by finite differences)



(Adapted from slides of James Tompkin, Brown University)

Illustration of Gradient Descent

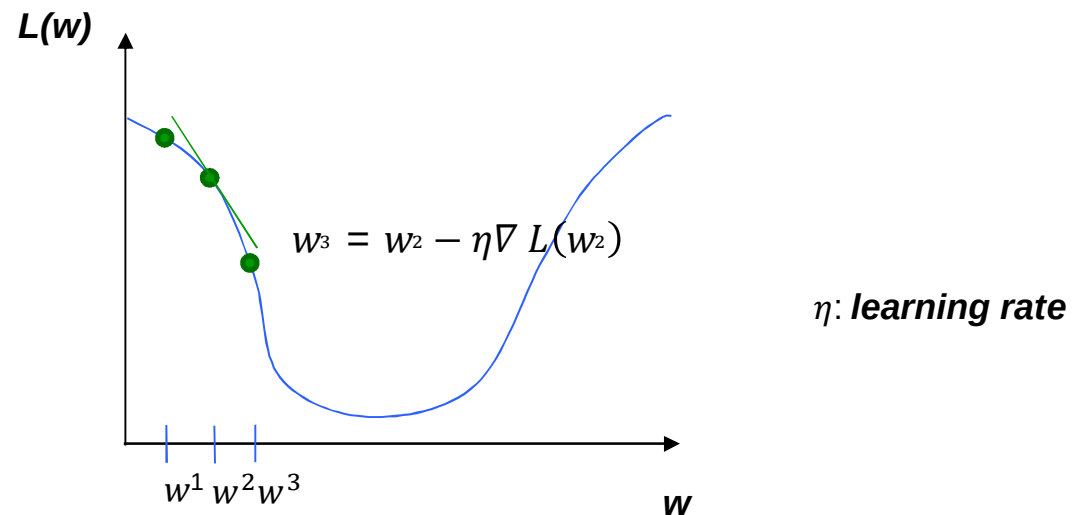
Move along parameter space in direction of negative gradient



(Adapted from slides of James Tompkin, Brown University)

Illustration of Gradient Descent

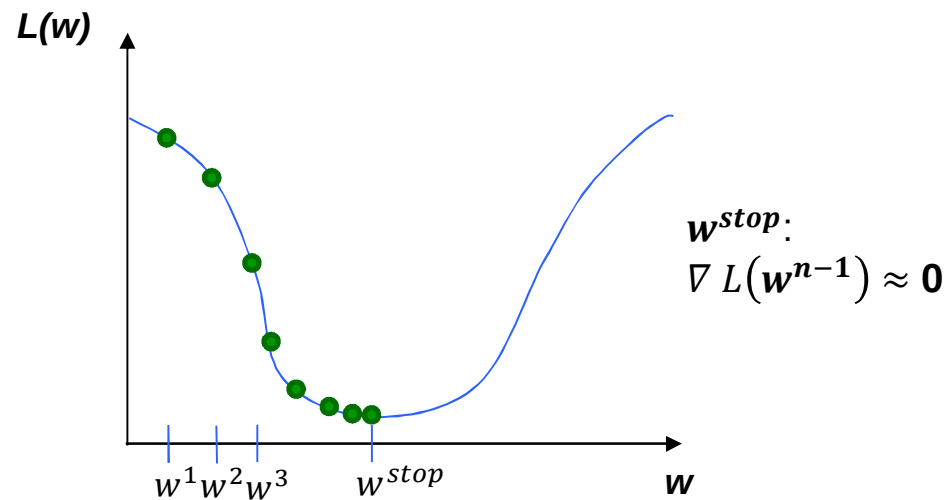
Move along parameter space in direction of negative gradient.



(Adapted from slides of James Tompkin, Brown University)

Illustration of Gradient Descent

Stop when w does not change or the change is very small.



How to Compute the Gradients

- ▶ Final layer weight update:

$$w_i = w_i - \eta \frac{\partial L}{\partial w_i}$$

- ▶ Gradient computation by chain rule:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

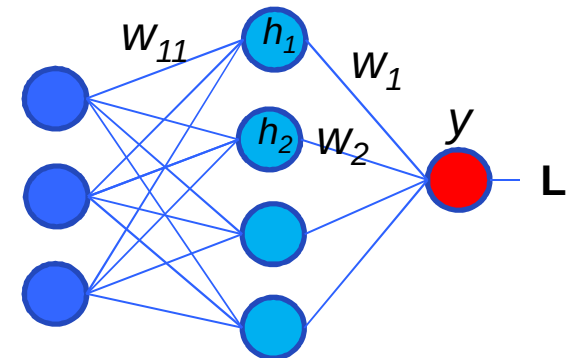
where $z = \sum_i w_i h_i$ and $y(z)$ is the activation function

- ▶ Example: assume:

- ▶ $L = \frac{1}{2}(t - y)^2$, where t is the true output (label)
- ▶ $y(z)$ is sigmoid: $y(z) = \sigma(z) = \frac{1}{1+e^{-z}}$

$$\frac{\partial L}{\partial y} = -(t - y), \quad \frac{\partial y}{\partial z} = \sigma(z)(1 - \sigma(z)), \quad \frac{\partial z}{\partial w_i} = h_i$$

$$\frac{\partial L}{\partial w_i} = (y - t)\sigma(z)(1 - \sigma(z)) h_i$$



How to Compute the Gradients

- ▶ Hidden layer weight update: $w_{11} = w_{11} - \eta \frac{\partial L}{\partial w_{11}}$
 - ▶ Gradient **backpropagation** by chain rule:

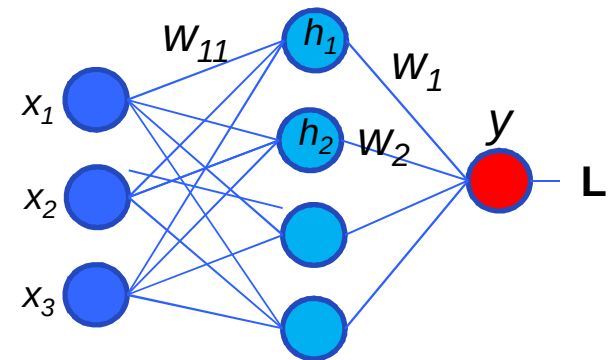
$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial h_1} \frac{\partial h_1}{\partial w_{11}}$$

- ▶ Assume sigmoid activation:

$$\frac{\partial y}{\partial h_1} = \frac{\partial y}{\partial z} \frac{\partial z}{\partial h_1} = \sigma(z)(1 - \sigma(z))w_1$$

$$\frac{\partial h_1}{\partial w_{11}} = \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{11}} = \sigma(z_1)(1 - \sigma(z_1))x_1$$

- ▶ $\frac{\partial L}{\partial y}$ and $\frac{\partial y}{\partial h_1}$ are **backpropagated** from the computation in the final layer



Backpropagation (*Cont'd*)

- ▶ Terminating condition:
Training stops when
 - ▶ all Δw_{ij} are smaller than a specified threshold, or
 - ▶ the percentage of examples misclassified in the previous epoch is below some threshold, or
 - ▶ a pre-specified number of **epochs** has expired.
 - ▶ An **epoch** is the training of the neural network with **all the training data in one iteration**

Backpropagation (*Cont'd*)

- ▶ Weight updates can be either case-based, epoch-based or mini-batch based
 - ▶ *Case updating*: weights are updated after presentation of each training example (a version of Stochastic Gradient Decent or SGD)
 - ▶ *Epoch updating*: weight increments are accumulated and are updated after all the training examples have been presented (Gradient Descent)
 - ▶ *Mini-batch updating*: present a randomly-selected subset of examples; weight increments are accumulated, and weights are updated after all the selected examples have been presented (min-batch SGD)

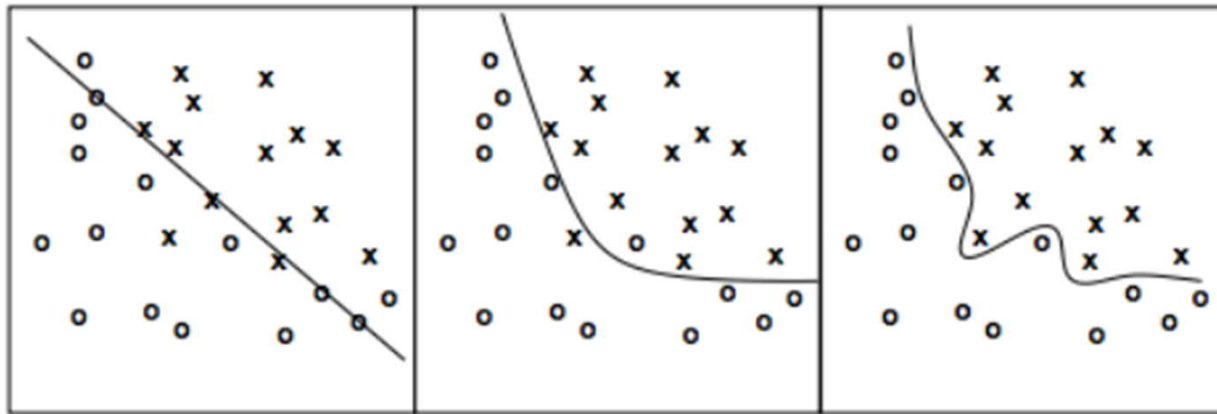
Min-batch Stochastic Gradient Descent

1. Initializes the model parameters $\mathbf{w}$ with small random values (e.g., ranging from -1.0 to 1.0, or -0.5 to 0.5)
2. Randomly shuffle the training examples into mini-batches
3. For each mini-batch B :
 - For each (x^i, y^i) in B ,
 - Propagate the inputs forward to obtain the output $\hat{y}^i$ using the current parameters $\mathbf{w}$:
 - Compute loss/error $L(\mathbf{w}, x^i)$ using y^i and $\hat{y}^i$:
 - Compute the partial gradient $\frac{\partial}{\partial w_j} L(\mathbf{w}, x^i)$ by **backpropagation**
 - Update each parameter w_j using:
$$w_j = w_j - \frac{\eta}{|B|} \sum_{x \in B} \frac{\partial}{\partial w_j} L(\mathbf{w}, x)$$
4. Go to Step 2 until $\mathbf{w}$ converges (or the number of epochs exceeds a threshold)

Outline

- ▶ *Introduction*
- ▶ *Types of neural network structures*
- ▶ *What happens inside a neuron?*
- ▶ *How to use a neural network for classification*
- ▶ *How to train a neural network*
 - ▶ *Backpropagation algorithm*
- ▶ **How to prevent overfitting**
- ▶ *Applications and history of neural networks*
- ▶ *Brief introduction to deep learning*

Overfitting

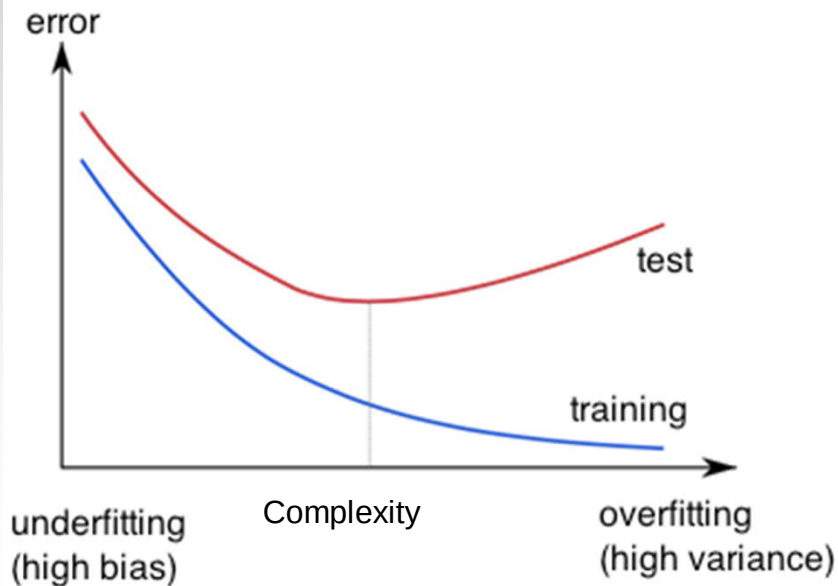


inadequate

good compromise

over-fitting

<http://wiki.bethanycrane.com/overfitting-of-data>



Learned hypothesis may **fit** the training data very well, even outliers (**noise**) but fail to **generalize** to new examples (test data)

https://www.neuraldesigner.com/images/learning/selection_error.svg

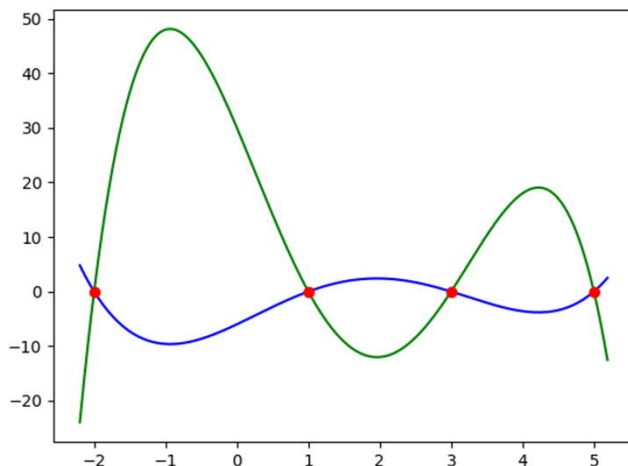
Prevent Overfitting

► Regularization

- Add regularization term to the objective function to penalize big weights (θ). For example,

$$L_{reg}(\theta) = L(\theta) + \lambda \sum_k \theta_k^2 \quad \leftarrow \begin{array}{l} \text{L2 regularization} \\ \text{(Squared L2 norm of } \theta) \end{array}$$

- Why regularizing the parameters helps to prevent overfitting?



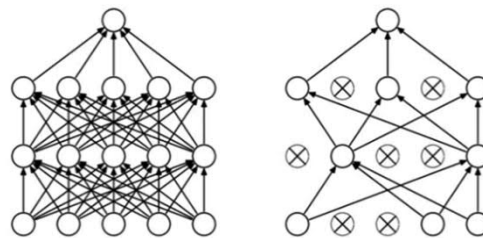
$$y_{\text{green}} = -x^4 + 7x^3 - 5x^2 - 31x + 30$$

$$y_{\text{blue}} = \frac{1}{5}x^4 - \frac{7}{5}x^3 + x^2 + \frac{31}{5}x - 6$$

Prevent Overfitting

▶ Dropout

- ▶ Randomly drop units (along with their connections) during training



▶ Benefits:

- ▶ Simplifies the network
 - ▶ A simpler network has less capacity to memorize the training data and is therefore less likely to overfit.
- ▶ Ensemble effect
 - ▶ Each time a different subset of neurons is dropped, similar to training a different “thinned” version of the network.
- ▶ Noise injection
 - ▶ Introduce a form of noise into the training process, preventing the model from fitting the data too closely

Prevent Overfitting

- ▶ Early stopping
 - ▶ Split the training data into building and validation sets.
 - ▶ Use validation error to decide when to stop training
 - ▶ Stop when monitored quantity has not improved after n subsequent epochs

Advantages of Neural Networks

- ▶ Solve problems that are too complicated for conventional techniques
 - ▶ can model both linear or non-linear functions
 - ▶ can do both classification and regression
- ▶ Able to learn from complicated or noisy data
- ▶ Problem solving (e.g., classification) with a neural network is fast

Limitations

- ▶ Training is slow
 - ▶ could take hours, days and even longer
- ▶ In general, few rules for setting up the parameters regarding the topology of the network
- ▶ Difficult for human to understand the learned target function
 - ▶ Effective “Black Boxes” whose rules of operation are completely unknown
- ▶ Important to have many known examples for training (inputs and related outputs)

Applications

- ▶ Computer Vision
- ▶ Speech Recognition
- ▶ Pattern Recognition
- ▶ Natural Language Processing (NLP)
- ▶ Financial prediction
- ▶ Screening Mortgage & Credit Applications
- ▶ Oil & Gas Exploration
- ▶ Medical Diagnosis
- ▶ Targeted Marketing

History of Neural Networks

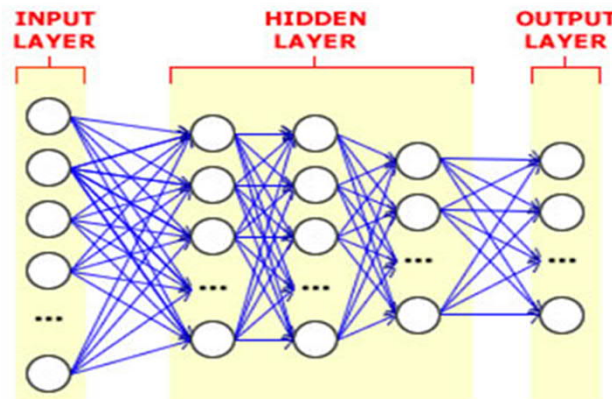
- ▶ 1943: McCulloch and Pitts proposed a model of a neuron --> Perceptron.
- ▶ 1960s: Widrow and Hoff explored Perceptron networks (which they called “Adelines”) and the delta rule.
- ▶ 1962: Rosenblatt proved the convergence of the perceptron training rule.
- ▶ 1969: Minsky and Papert showed that the Perceptron cannot deal with nonlinearly-separable data sets---even those that represent simple function such as X-OR.
- ▶ 1970-1985: Very little research on Neural Nets
- ▶ 1986: Invention of Backpropagation [Rumelhart and McClelland, but also Parker and earlier on: Werbos] which can learn from nonlinearly-separable data sets.
- ▶ Since 1986: A lot of research in Neural Nets!
- ▶ In 2006, Geoff Hinton, et al., showed an effective way to train a **deep neural network (DNN)** by
 - ▶ effectively pre-training one layer at a time in an unsupervised way and
 - ▶ then fine-tuning it using supervised backpropagation
- ▶ Since then, many DNN architectures have been proposed and deep learning is the state-of-the-art machine learning technique.

Outline

- ▶ Introduction
- ▶ Types of neural network structures
- ▶ What happens inside a neuron?
- ▶ Activation functions
- ▶ Input and output design
- ▶ How to train a neural network
 - ▶ Backpropagation algorithm
- ▶ How to prevent overfitting
- ▶ Applications and history of neural networks
- ▶ **Brief introduction to deep learning**

Deep Neural Networks

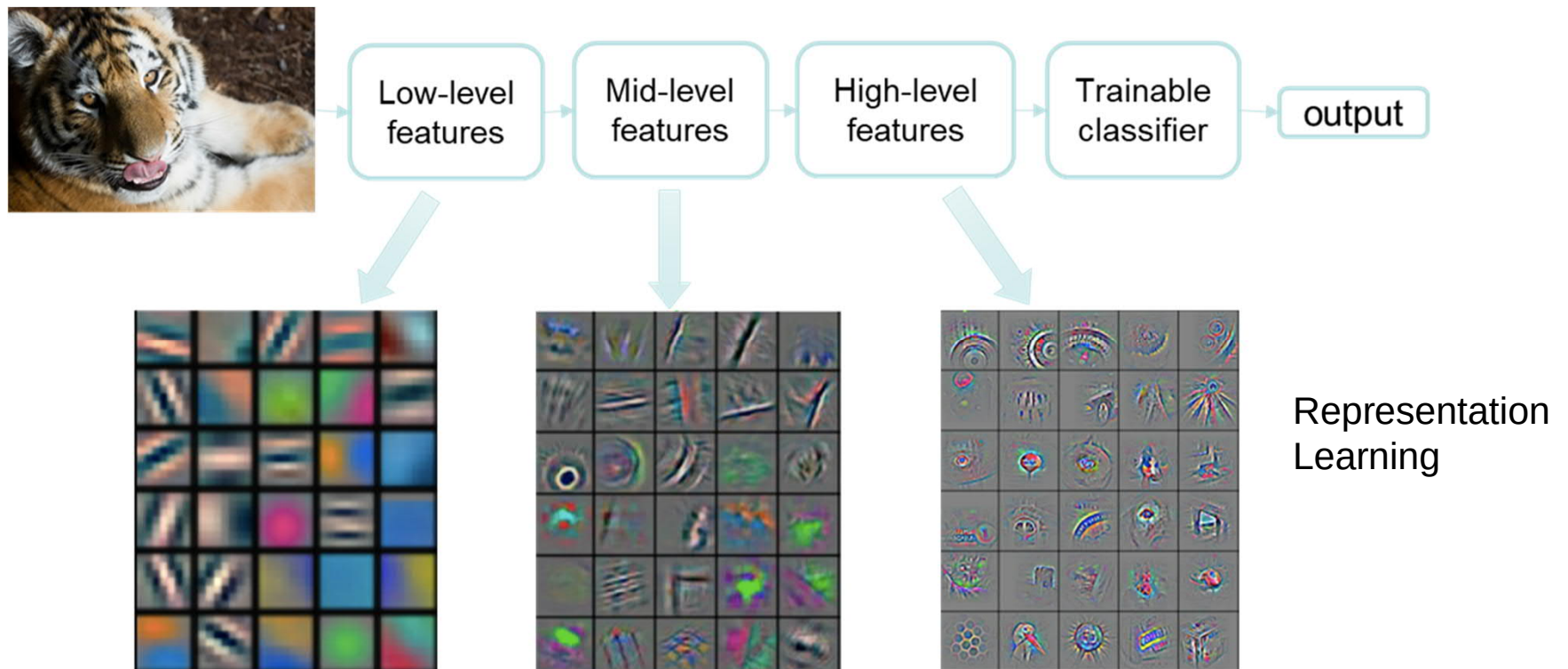
- ▶ Neural network with more than one hidden layer of nodes between input and output



- ▶ Providing breakthrough results in
 - ▶ Image classification
 - ▶ Speech recognition
 - ▶ Natural language processing
 - ▶

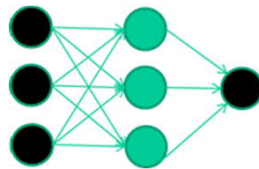
Deep Neural Networks

- ▶ Why is it generally better than other methods?
 - ▶ Seek to learn rich hierarchical representations (i.e. features) automatically through multiple stage of feature learning process.

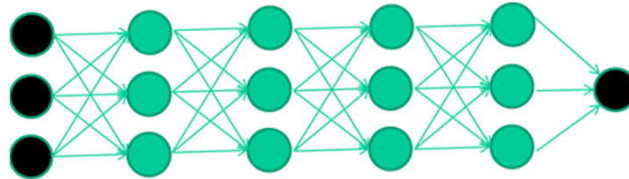


Deep Neural Networks

- ▶ The previous NN training algorithm is
 - ▶ good for training networks with 1 or 2 hidden layers



- ▶ Not good at learning networks with more hidden layers.

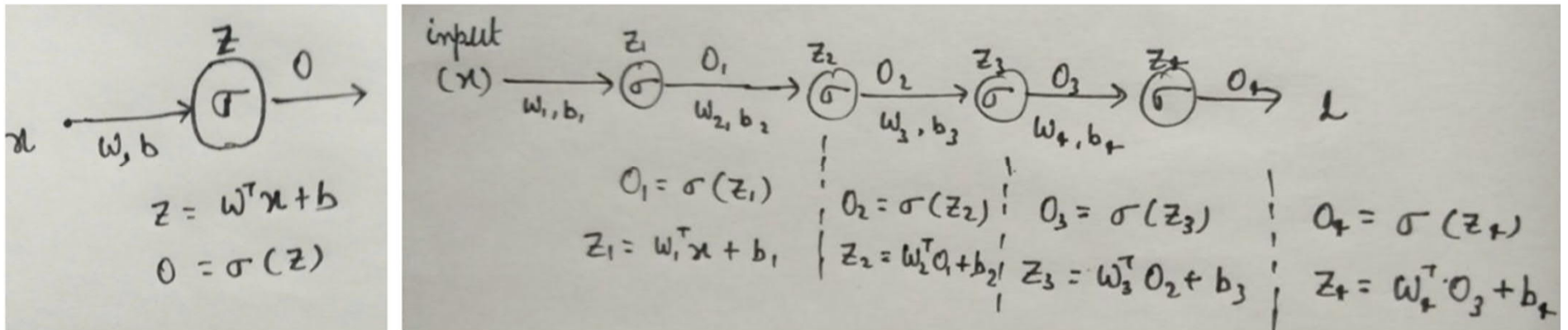


Perform worse than NN
with 1 or 2 layers

- ▶ Reasons:
 - ▶ Vanishing and exploding gradients problems
 - ▶ Many basins in loss function exist and easily get stuck in poor local minima

Gradient Vanishing Problem

- ▶ Gradients of the loss function with respect to the weights in the earlier layers of the network become **extremely small** or **vanish**



- ▶ Recall $w_i = w_i + \Delta w_i$ and $\Delta w_i = -\eta \frac{\partial L}{\partial w_i}$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial O_4} \frac{\partial O_4}{\partial O_3} \frac{\partial O_3}{\partial O_2} \frac{\partial O_2}{\partial O_1} \frac{\partial O_1}{\partial w_1} ; \text{ where, } O_i = \sigma(z_i) \quad z_i = w_i o_{i-1} + b_i$$

- ▶ Derivatives of Sigmoid range (0,0.25) and derivatives of Tahn range (0,1)
- ▶ Problem:
 - ▶ Weights in the earlier layers are very hard or slow to train, leading to a stagnation in the learning process.

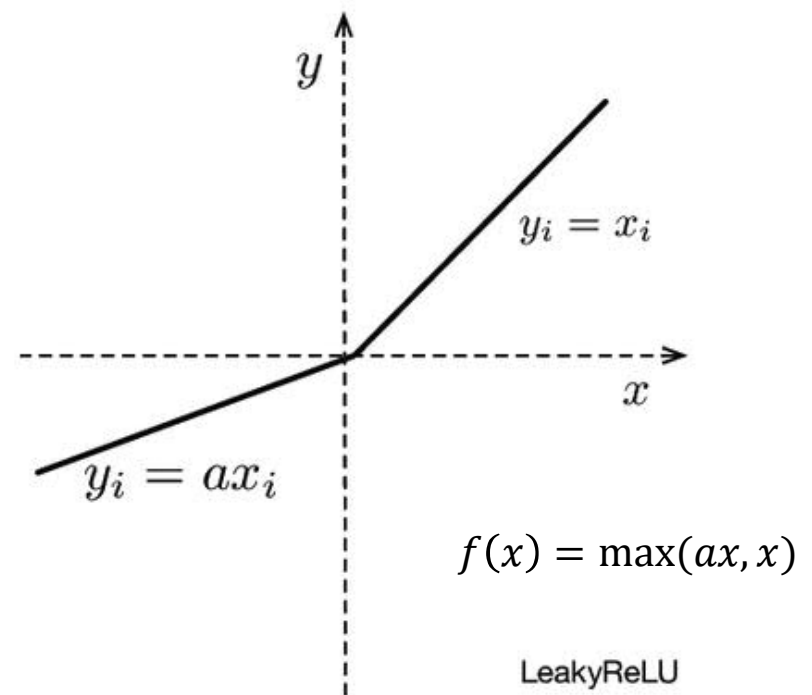
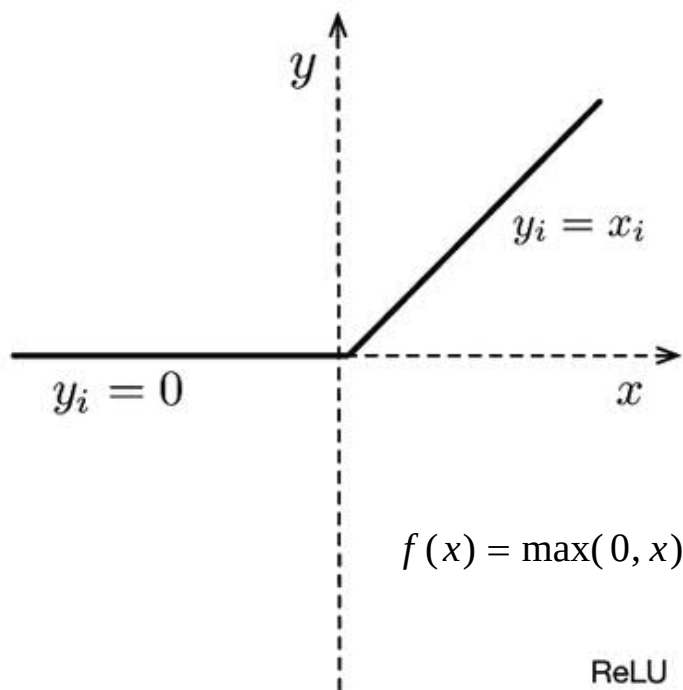
Gradient Vanishing Problem

- ▶ Causes of the problem:
 - ▶ Activation Functions:
 - ▶ Sigmoid and tanh functions have gradients in the range $(0, 0.25)$ and $(0, 1)$, respectively. When these gradients are multiplied during backpropagation, they can diminish exponentially with each layer.
 - ▶ Deep Architectures:
 - ▶ In very deep networks, the repeated multiplication of small gradients can lead to an exponential decrease in the gradient magnitude as it is propagated back through the layers.
 - ▶ Poor Weight Initialization:
 - ▶ If the weights are initialized poorly, it can exacerbate the vanishing gradient problem. For instance, weights that are too small can lead to small gradients during backpropagation.

Solutions to Gradient Vanishing Problem

- ▶ Activation functions:

- ▶ Use ReLu or its variants (e.g., Leaky ReLu, Parametric ReLu, etc.)



- ▶ Leaky ReLu:

- ▶ avoid the "dying ReLU" problem, where some neurons can become permanently inactive during training.

Solutions to Gradient Vanishing Problem

- ▶ Gradient Clipping:
 - ▶ Force the gradient values (element-wise) to a specific minimum or maximum value if the gradient exceeds an expected range.
- ▶ Network Architectures:
 - ▶ Skip Connections/Residual Networks (ResNets):
 - ▶ Architectures like ResNets use skip connections to allow gradients to flow through the network more directly, mitigating the vanishing gradient problem.
 - ▶ Use of Gated Units in RNNs:
 - ▶ In recurrent neural networks (RNNs), gated units like Long Short-Term Memory (LSTM) cells and Gated Recurrent Units (GRUs) are designed to address the vanishing gradient problem by incorporating mechanisms that allow gradients to flow unchanged.

Gradient Exploding Problem

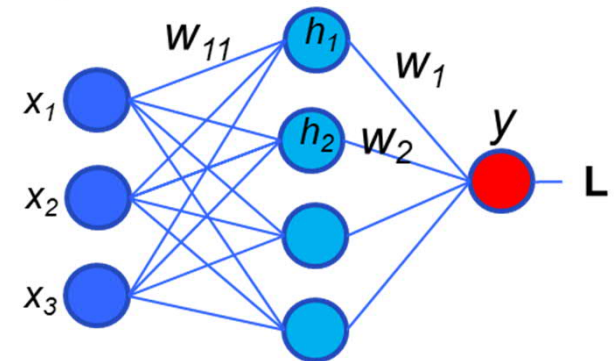
- ▶ Gradients are too large leading to very large updates to the network's weights, causing
 - ▶ the model to become unstable
 - ▶ the loss function to diverge (i.e., loss increases)
- ▶ Gradient backpropagation by chain rule:

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial h_1} \frac{\partial h_1}{\partial w_{11}}$$

where

$$\frac{\partial y}{\partial h_1} = \frac{\partial y}{\partial z} \frac{\partial z}{\partial h_1} = \frac{\partial y}{\partial z} w_1$$

$$\frac{\partial h_1}{\partial w_{11}} = \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{11}} = \frac{\partial h_1}{\partial z_1} x_1$$



- If w_i or x_i is large, the partial derivative values can be greater than 1.
- If all weights are large values and the network is deep, gradients can explode.

Gradient Exploding Problem

- ▶ Symptoms:
 - ▶ NaN (Not a Number) values in the gradients or weights.
 - ▶ A very high loss at the beginning of training, or a loss that becomes very high suddenly.
 - ▶ An unstable model where the output for a given input changes dramatically over training iterations.

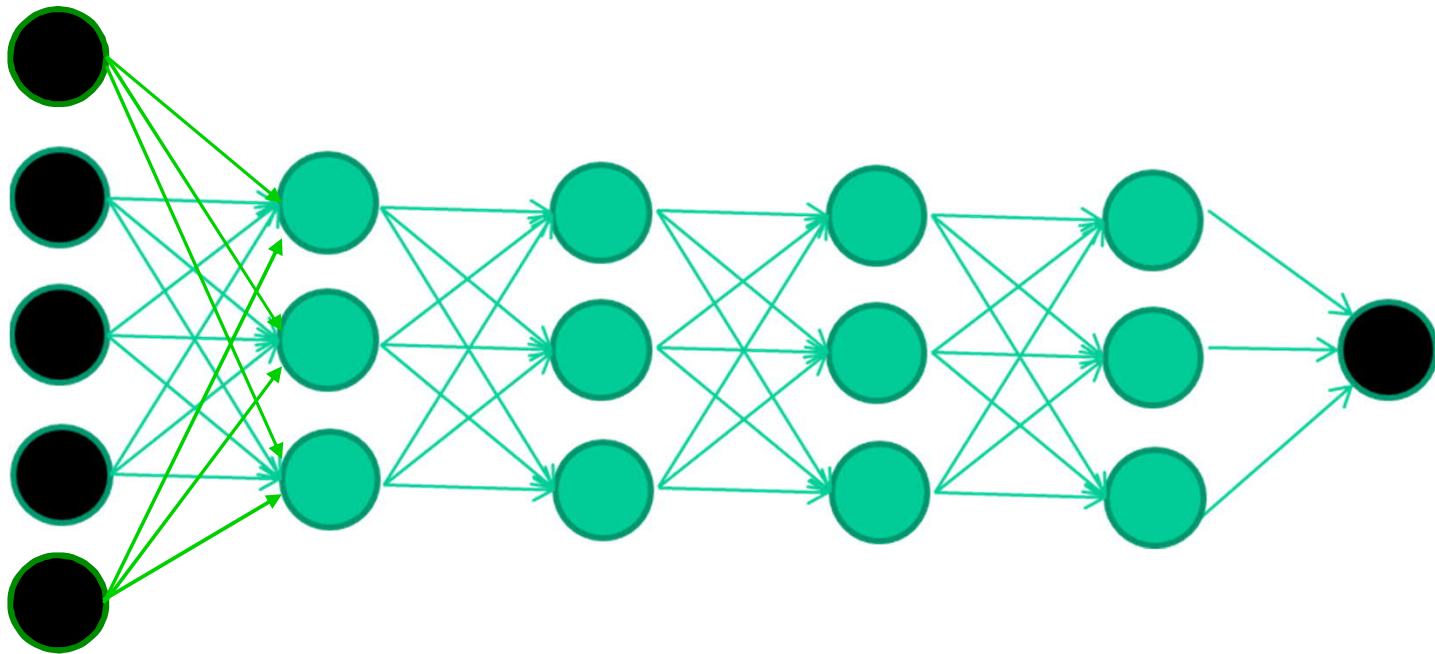
Solutions to Gradient Exploding Problem

- ▶ Gradient Clipping:
 - ▶ Clipping the gradients during backpropagation to ensure they do not exceed a certain threshold.
- ▶ Gradient Scaling:
 - ▶ Normalizing the error gradient vector so that the vector norm (magnitude) equals a predefined value, such as 1.0
- ▶ Weight Initialization:
 - ▶ Proper initialization of network weights to small values can help prevent gradients from becoming too large.
- ▶ Batch Normalization:
 - ▶ Normalize the outputs of an intermediate layer over a batch of training data.

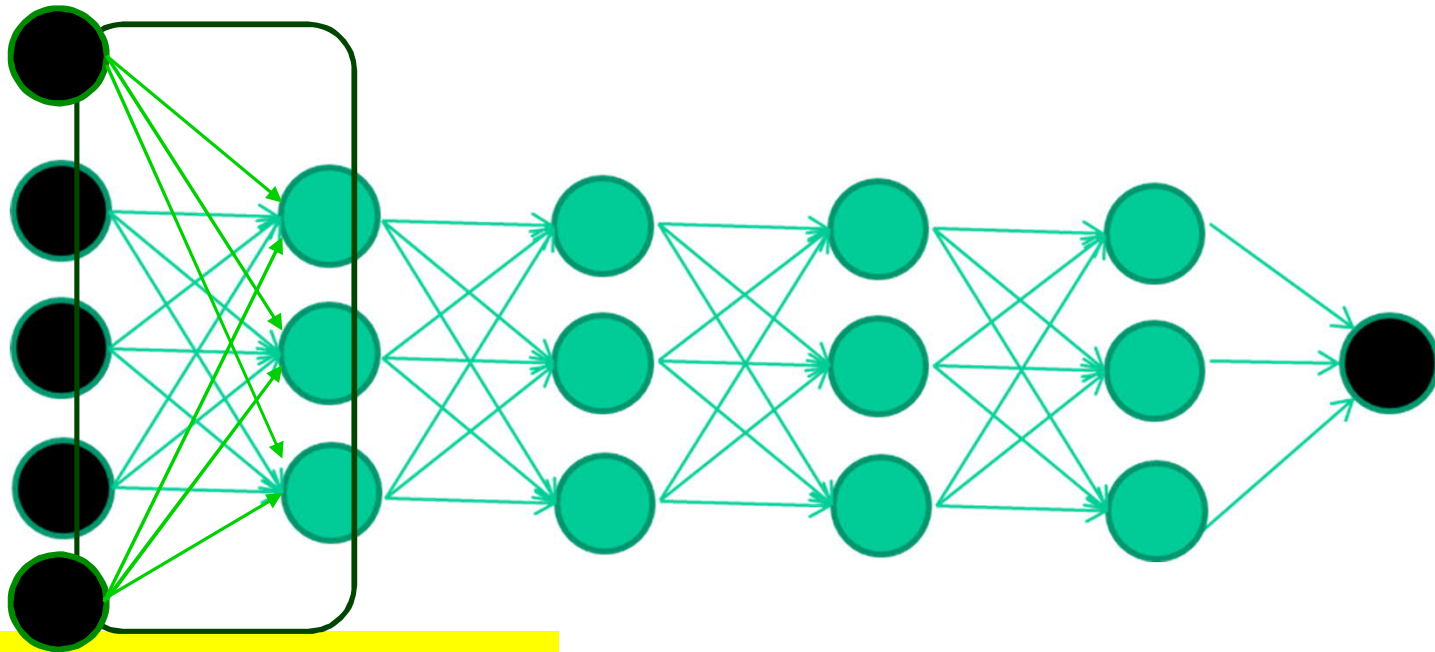
A Solution for Stuck on Poor Local Minimum

- ▶ A trick to avoid **getting stuck on a poor local minimum**:
 - ▶ Use unsupervised learning to initialize (pre-train) weights
 - ▶ Weights are initialized layer by layer
 - ▶ Each layer is trained using an auto-encoder
 - ▶ Two such early tricks (used popularly in 2006 – early 2010s)
 - ▶ Stacked Autoassociator or autoencoder Network to pre-train a deep NN.
 - ▶ Stacked Restricted Boltzmann machines (RBM) to pre-train a deep believe network

Pre-train Multi-layer NN Layer-by-layer

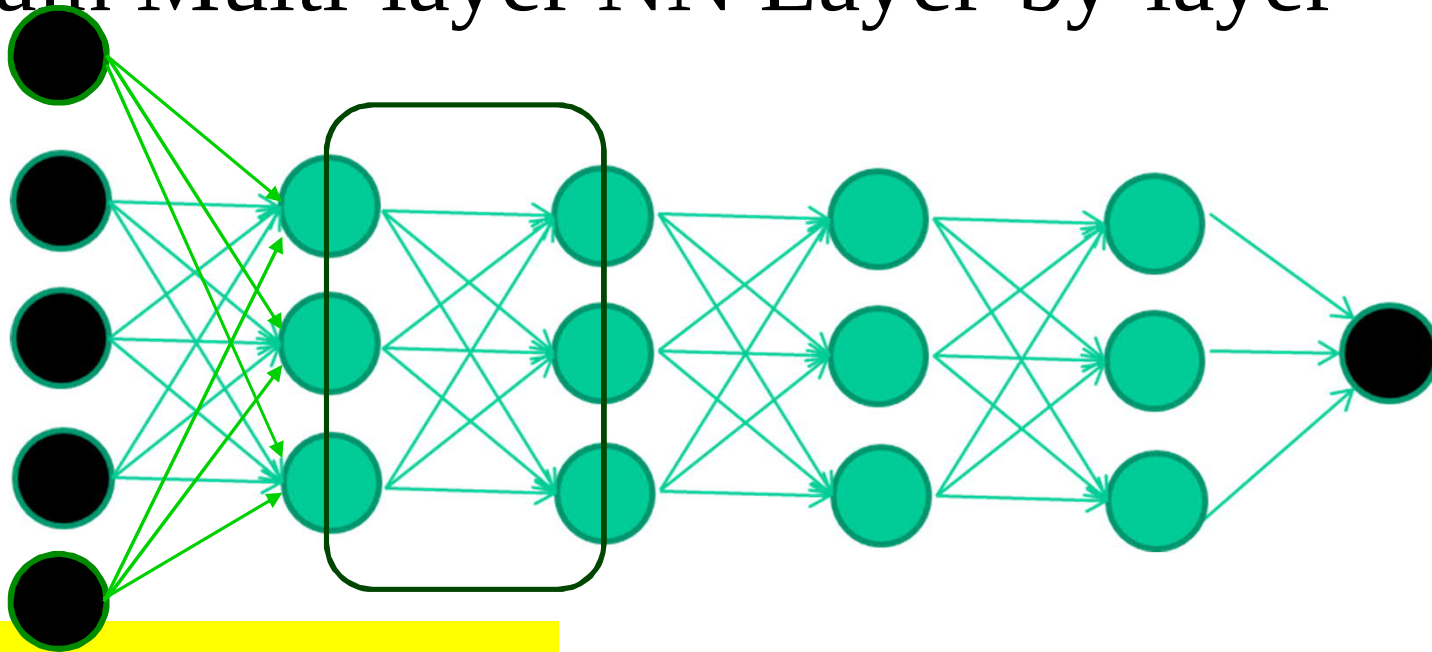


Pre-train Multi-layer NN Layer-by-layer



Train this layer first

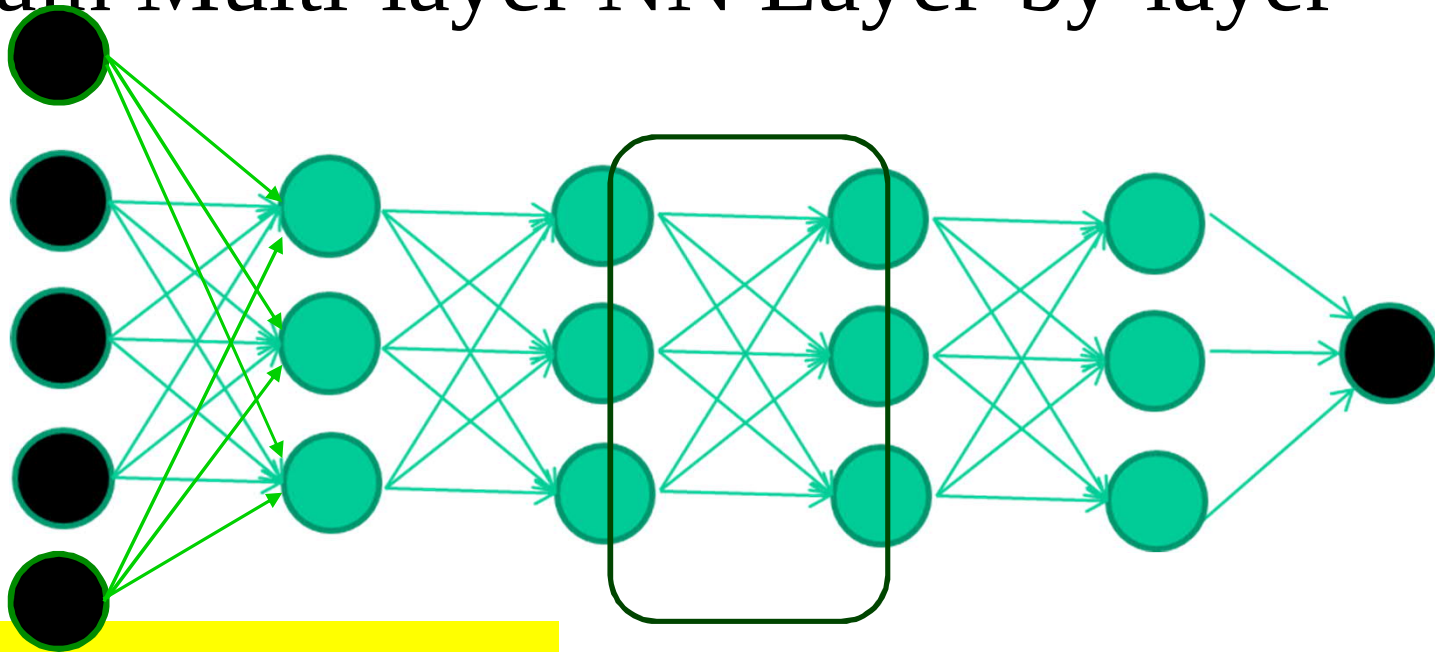
Pre-train Multi-layer NN Layer-by-layer



Train this layer first

then this layer

Pre-train Multi-layer NN Layer-by-layer

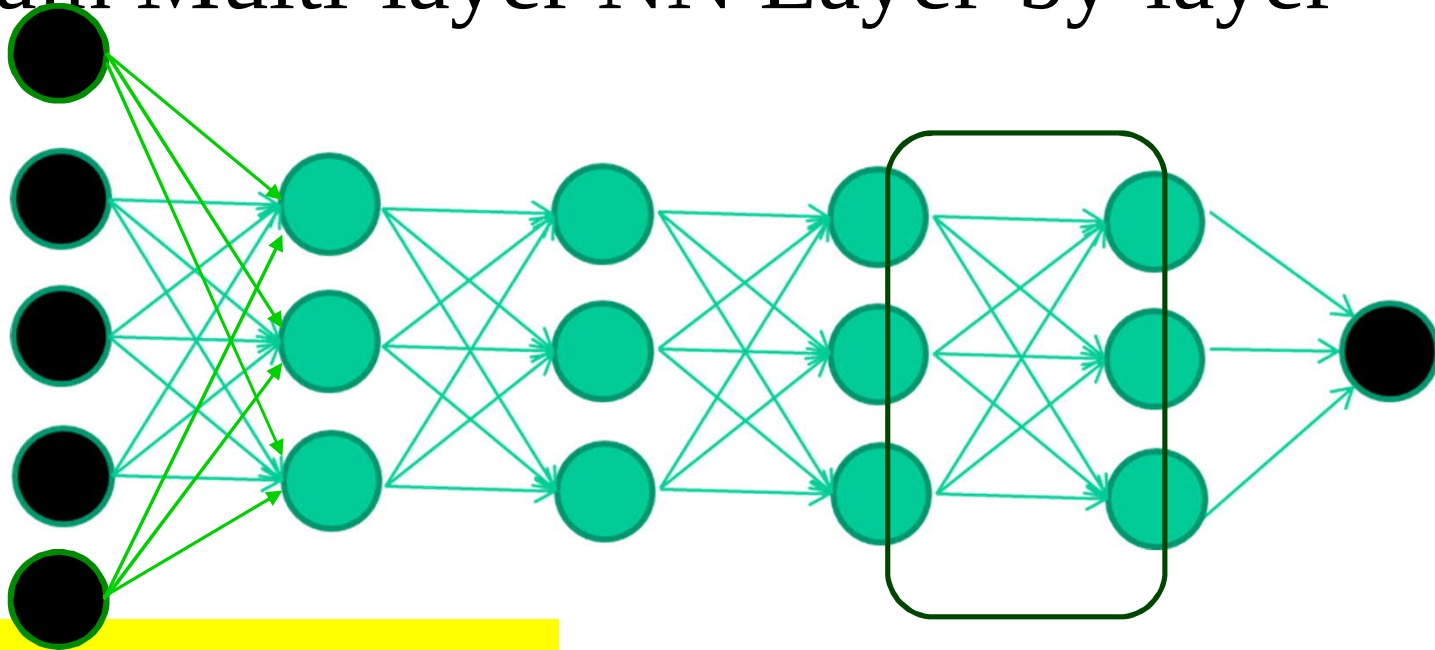


Train this layer first

then this layer

then this layer

Pre-train Multi-layer NN Layer-by-layer



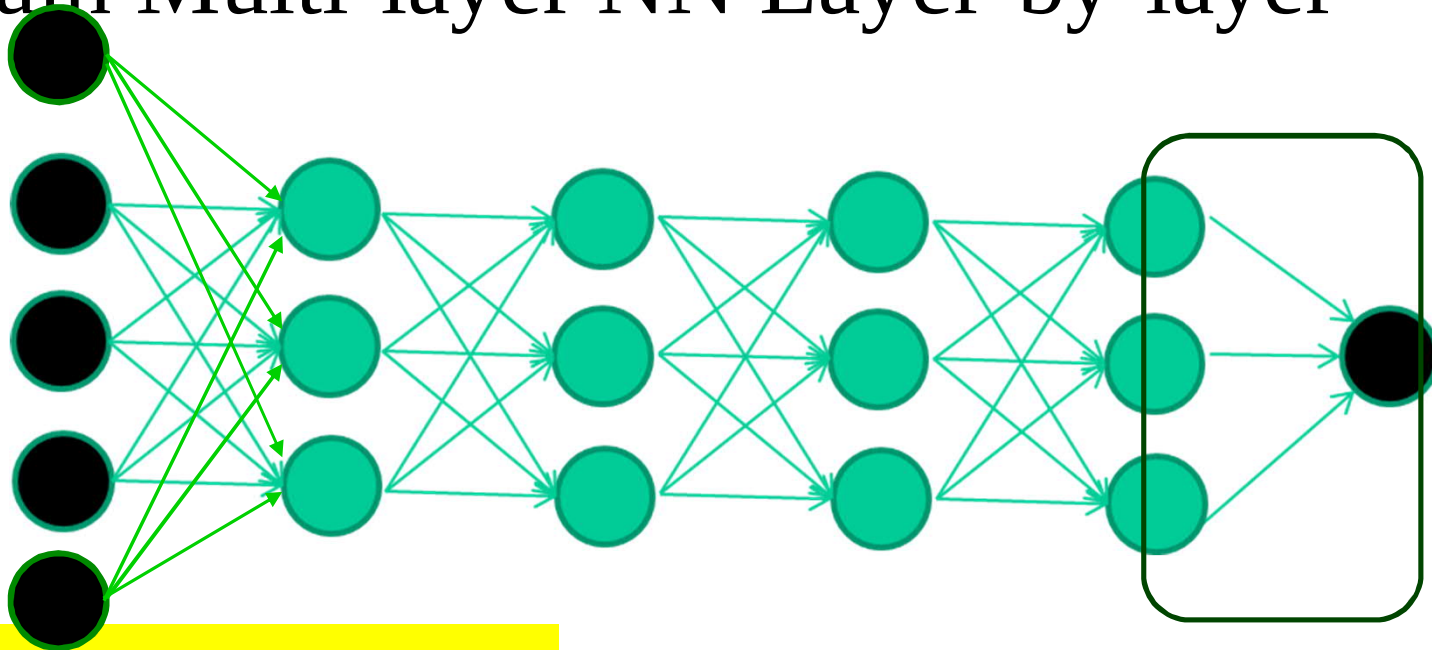
Train this layer first

then this layer

then this layer

then this layer

Pre-train Multi-layer NN Layer-by-layer



Train this layer first

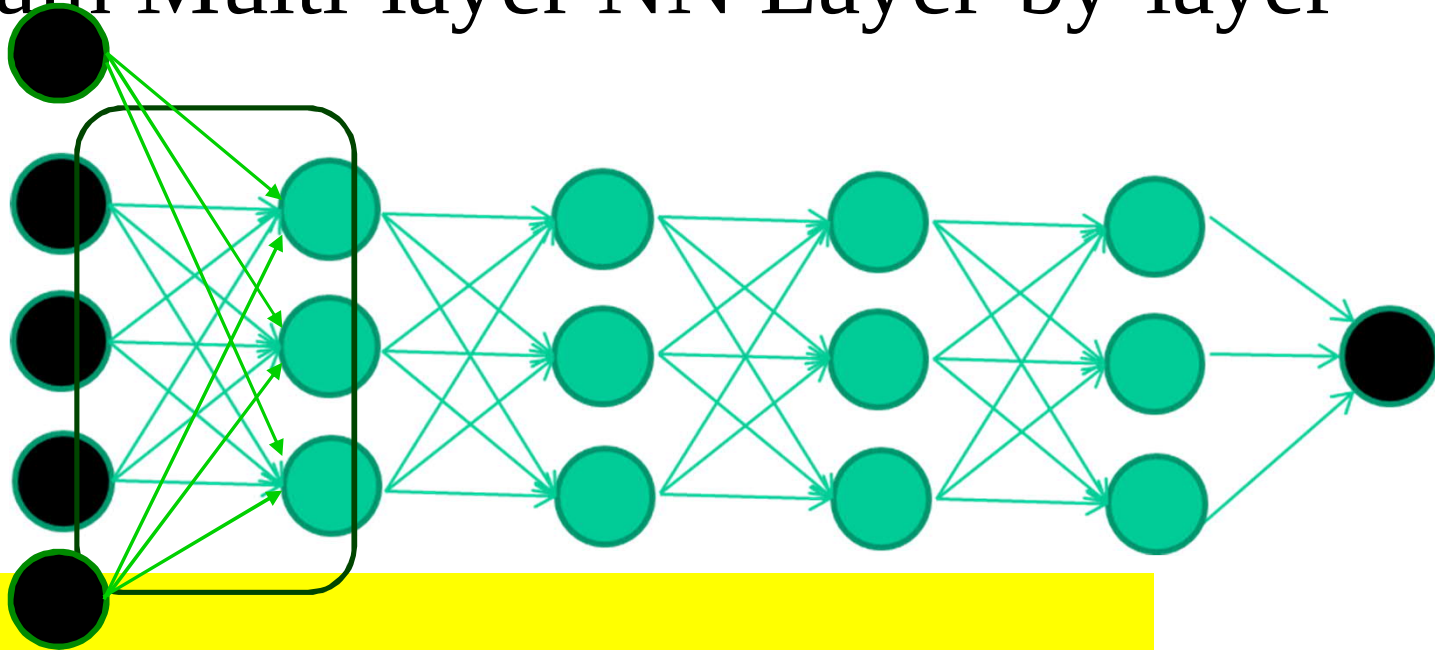
then this layer

then this layer

then this layer

finally this layer

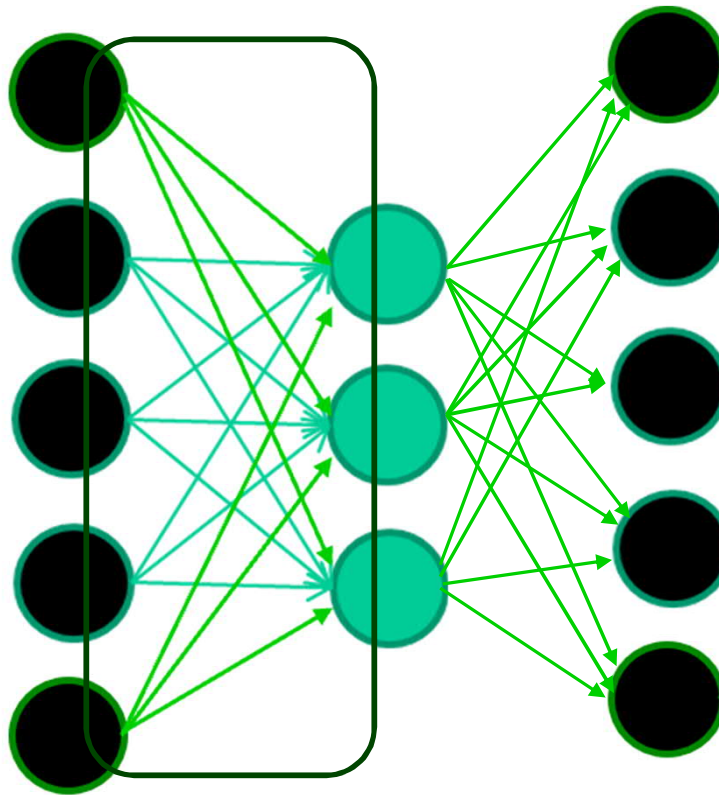
Pre-train Multi-layer NN Layer-by-layer



EACH of the (non-output) layers is trained to be, e.g., an **auto-encoder**

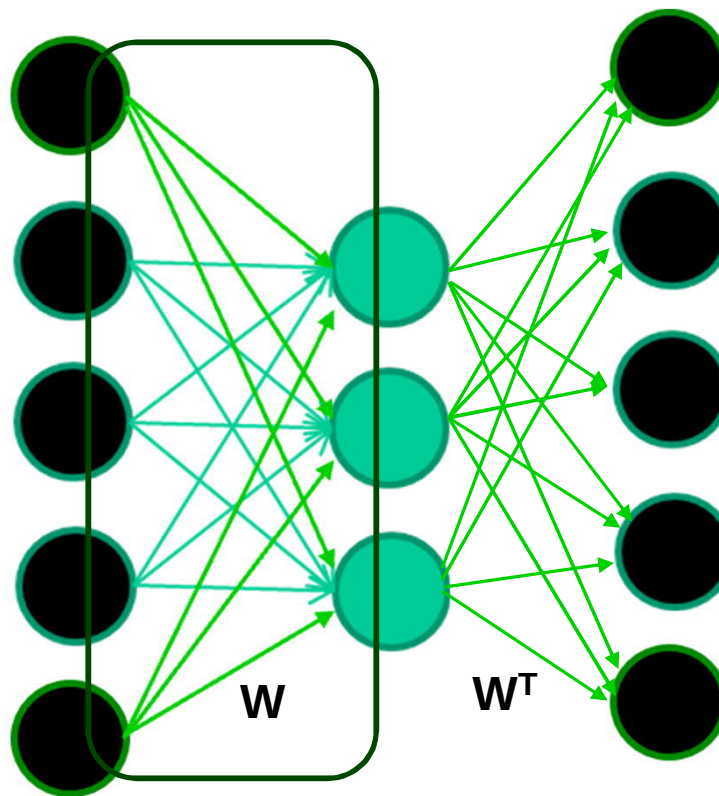
Basically, it is forced to learn good features that describe what comes from the previous layer

Auto-encoder



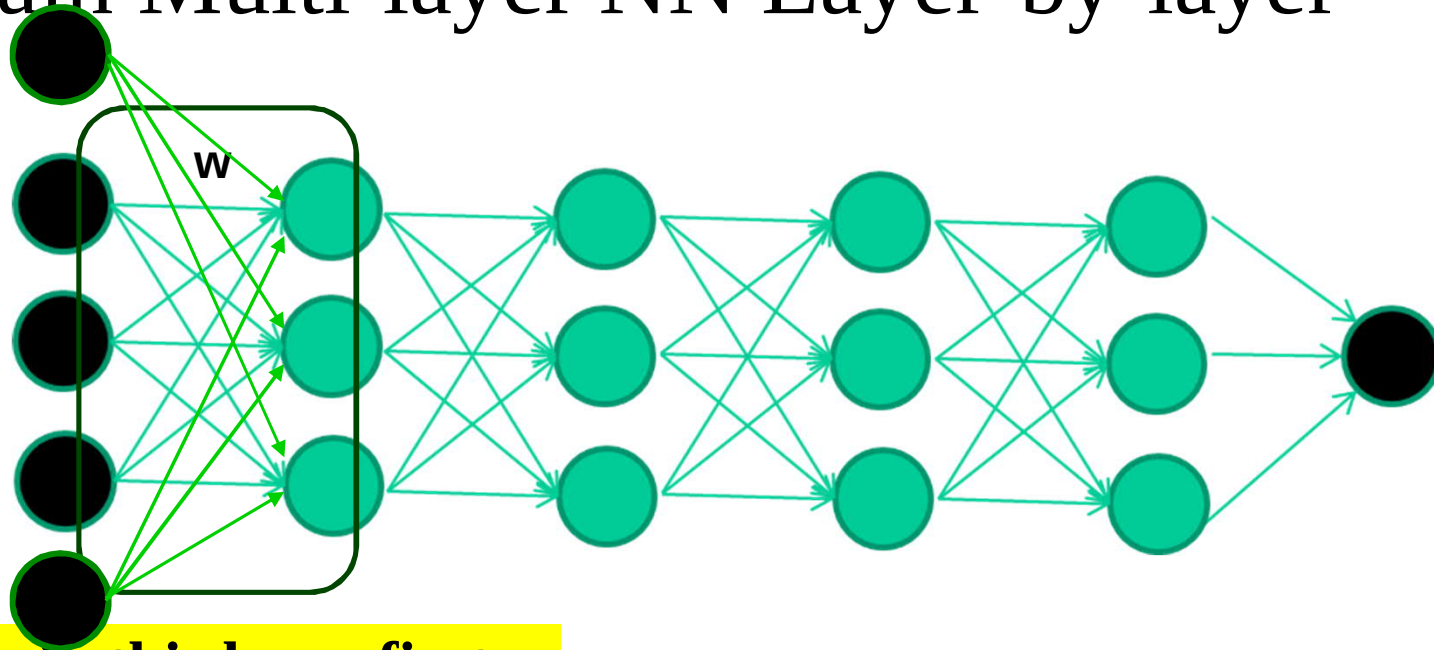
An auto-encoder is trained, with an absolutely standard weight-adjustment algorithm to reproduce the input

Auto-encoder



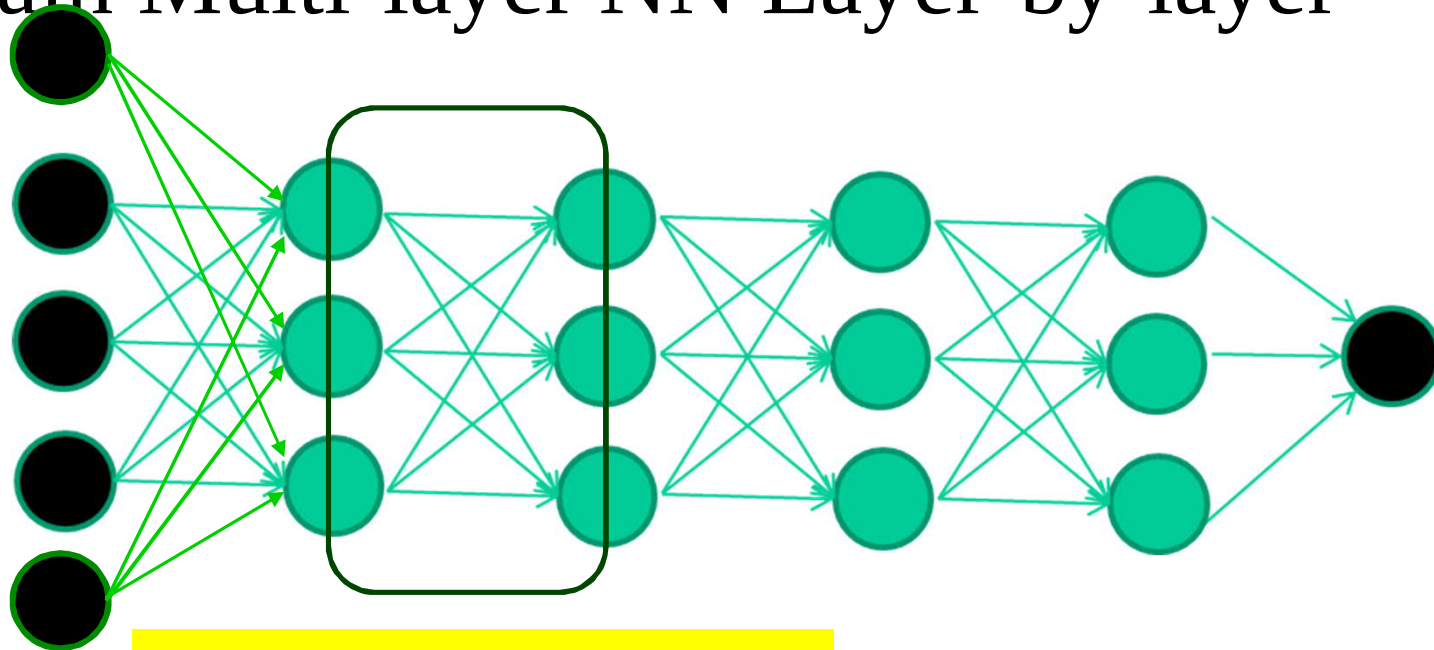
By making this happen with (many) fewer units than the inputs, this forces the ‘hidden layer’ units to become good feature detectors

Pre-train Multi-layer NN Layer-by-layer



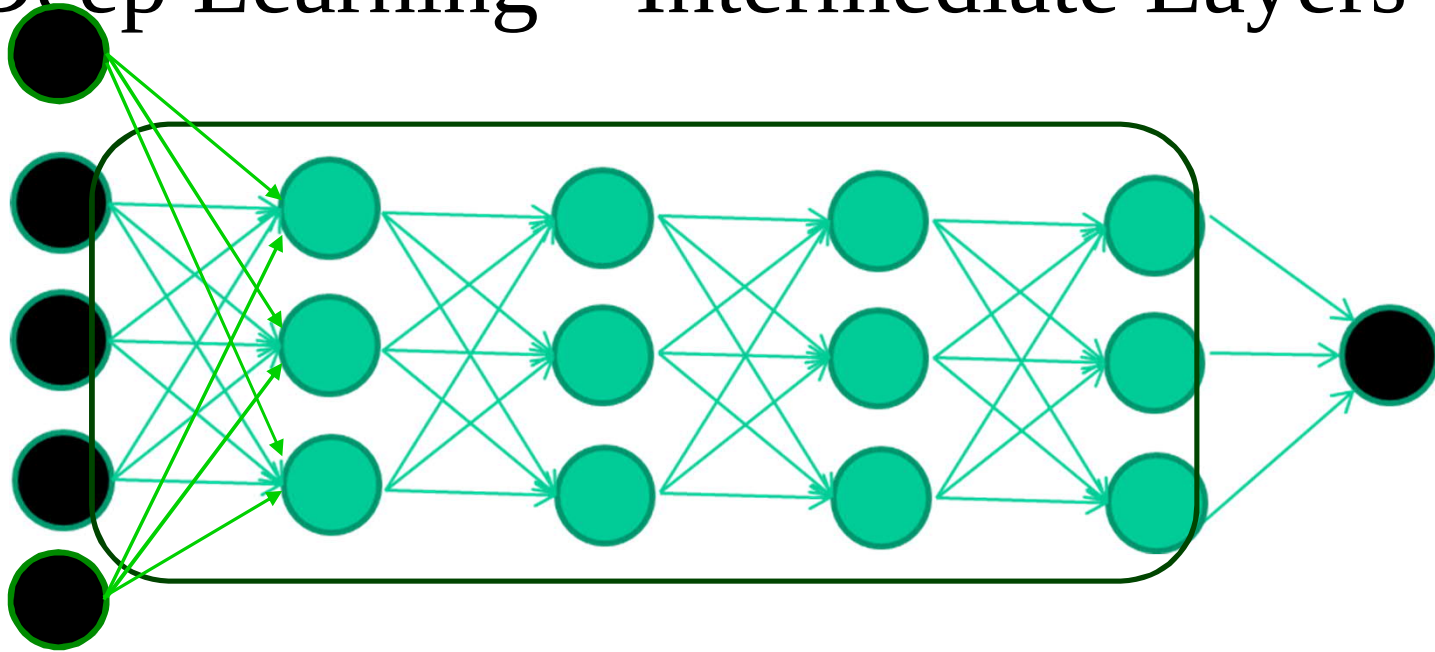
**Train this layer first
using auto-encoder**

Pre-train Multi-layer NN Layer-by-layer



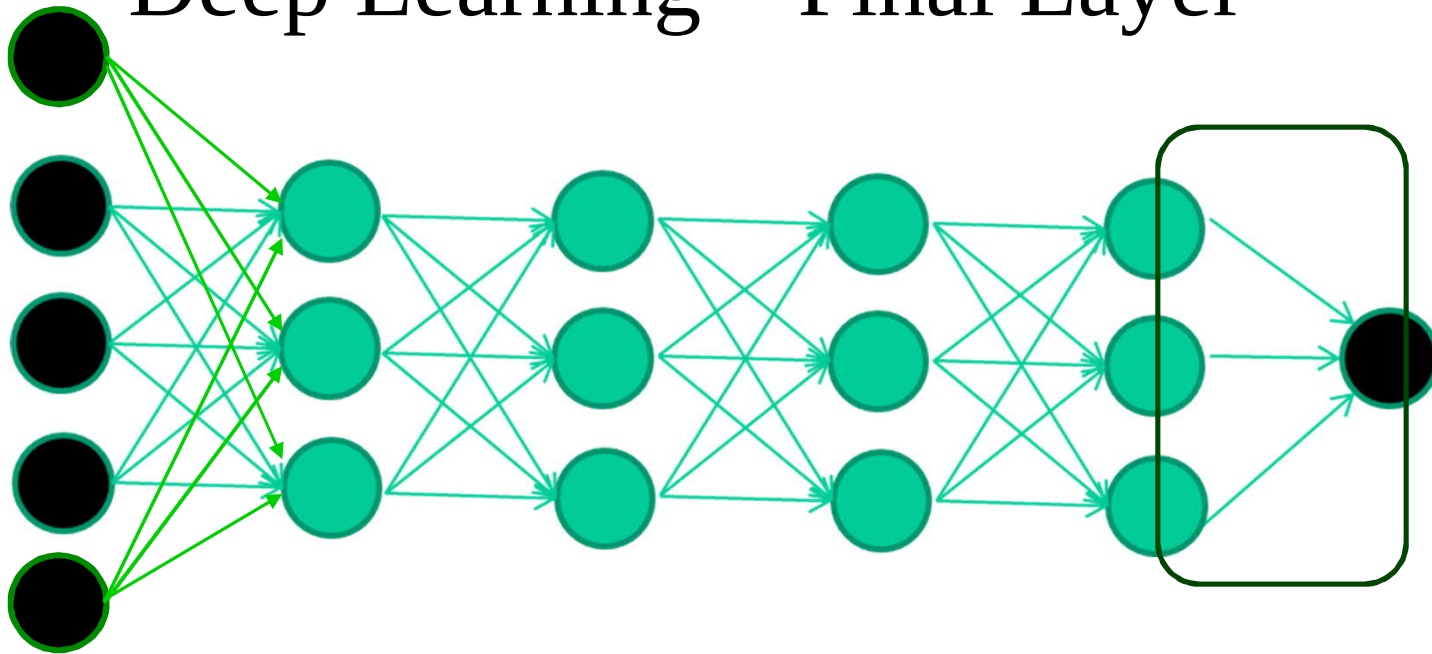
**then training this layer
using auto-encoder**

Deep Learning – Intermediate Layers



Intermediate layers are each trained to be auto encoders (or similar)

Deep Learning – Final Layer



Final layer is trained to predict class labels

- The weights for the previous layers are initialized with the weights trained by auto-encoders
- The final layer weights are initialized randomly
- The final training (with labeled data) updates all the weights using back-propagation.

Deep Learning

- ▶ That's an early idea (2006)
- ▶ There have been
 - ▶ Different kinds of autoencoders, variations on architectures
 - ▶ Different weight initialization techniques, such as transfer learning, self-supervised learning, etc.
 - ▶ Many types of deep neural network structures
 - ▶ ...
- ▶ Very fast growing area ...

Deep Learning Architectures

- ▶ Convolutional neural networks
- ▶ Residual neural networks
- ▶ Recurrent neural networks
 - ▶ LSTM, GRU
- ▶ Sequence-to-sequence model
- ▶ Transformer
- ▶ Big language models
 - ▶ BERT, GPT-3, SBERT, T5, BART, MT-NLG, ChatGPT, GPT-4